TABLE OF CONTENTS

1. INTRODUCTION 1

1.1 STATEMENT OF OBJECTIVES 1

1.2 SCOPE OF THE PROJECT 2

2. LITERATURE STUDY 3

3. SOFTWARE REQUIREMENT ANALYSIS 5

3.1 HARDWARE SPECIFICATION 5

3.2 SOFTWARE SPECIFICATION 6

3.3 ABOUT THE SOFTWARE 7

3.4 FEATURES 8

4. SYSTEM ANALYSIS 10

4.1 REQUIREMENT SPECIFICATION 10

4.2 CHARACTERISTICS OF EXISTING SYSTEM 11

4.3 FEASIBILITY STUDY 12

4.4 SOFTWARE REQUIREMENT SPECIFICATION (SRS) 13

5. SYSTEM DESIGN 15

5.1 SYSTEM ARCHITECTURE 15

5.2 USE CASE DIAGRAM 17

5.3 ACTIVITY DIAGRAM 18

5.4 CLASS DIAGRAM 19

5.5 COMPONENT DIAGRAM 20

5.6 DATA FLOW DIAGRAM (DFD) 21

5.7 MATHEMATICAL MODELING OF NEURAL ARCHITECTURES 22

6. SYSTEM IMPLEMENTATION 25

6.1 MODULE DESCRIPTION 25

6.2 VALIDATION CHECKS 27

7. SYSTEM TESTING 30

7.1 UNIT TESTING MODULES 30

7.2 INTEGRATED TESTING 31

7.3 TEST CASES 33

8. RESULT AND CONCLUSION 35

8.1 RESULT 35

8.2 FUTURE ENHANCEMENTS 36

9. APPENDICES 38

9.1 PLAGIARISM CERTIFICATE 38

9.2 SCREENSHOTS 39

9.3 SOURCE CODE 44

9.4 USER DOCUMENTATION 64

9.5 GLOSSARY 68

10. BIBILIOGRAPHY 71

10.1 Book References 71

10.2 Journal References 72

ABSTRACT

Traditional psychopathology assessments rely heavily on subjective clinical interviews, which are often susceptible to patient bias and the conscious masking of true emotional states. To address the need for objective, continuous emotional monitoring, this project presents a Multi-modal Psychopathology Assessment system designed for robust state-of-mind prediction. The proposed architecture fuses visual and acoustic modalities to provide a comprehensive evaluation of a subject's affective state, mitigating the false positives inherent in single-modality systems. The visual channel employs a custom 2D Convolutional Neural Network (CNN) trained on the FER-2013 dataset to classify seven distinct facial expressions extracted from video frames. Concurrently, the auditory channel utilizes an advanced, hybrid 1D CNN-LSTM network trained on the RAVDESS dataset. Addressing early developmental limitations, the robust auditory model incorporates targeted data augmentation—such as noise injection, pitch shifting, and time stretching—and utilizes Mel-frequency cepstral coefficients (MFCCs) to capture long-term temporal and prosodic dependencies across eight acoustic emotion classes. Furthermore, the system integrates a K-Means clustering algorithm for speaker diarization, allowing the application to isolate and track emotions for specific individuals in multi-speaker environments. These dual independent streams are synchronized within a Streamlit-based web application. The integration hub utilizes a late-fusion algorithmic strategy, actively comparing synchronous visual and auditory predictions to calculate emotional congruence and flag incongruent affect (e.g., a smiling face paired with a stressed voice). The final output is rendered through an interactive, browser-based multimodal player and dynamic Plotly visualizations. Ultimately, this platform equips mental health professionals with detailed, chronological insights regarding a patient's emotional fluctuations, functioning as an impartial clinical decision tool during telehealth consultations and enabling long-term psychological, psychiatric and mental health tracking to support comprehensive patient wellness.

LIST OF FIGURES

S NO	FIGURE NO	FIGURE NAME	PAGE NO
1	5.1.1	System Architecture	16
2	5.2.1	Use Case Diagram	17
3	5.3.1	Activity Diagram	18
4	5,4,1	Class Diagram	19
5	5,5,1	Component Diagram	20
6	5.6.1	Data Flow Diagram	21

LIST OF TABLES

S NO	TABLE NO	FIGURE NAME	PAGE NO
1	7.3.1	Test Cases	34

LIST OF SCREENSHOTS

S NO	PHOTO NO	PHOTO NAME	PAGE NO
1	9.1.1	PLAGIARISM CERTIFICATE	38
2	9.2.1	INITIAL DASHBOARD	39
3	9.2.2	ACTIVE PROCESSING STATE	40
4	9.2.3	MULTIMODAL PLAYER	41
5	9.2.4	MODAL FUSION TIMELINE	42
6	9.2.5	FULLSCREEN EMOTION SCATTER PLOT	43

1. INTRODUCTION

Traditional methods for assessing psychopathology and mental state rely heavily on subjective clinical interviews and self-reported questionnaires. While these traditional diagnostic methods are effective, they inherently lack continuous monitoring capabilities and are highly susceptible to patient bias. Patients often consciously or subconsciously mask their true feelings, making it difficult for clinicians to gauge their authentic psychological state. Recent advancements in Affective Computing offer a pathway to objectify these assessments by analyzing non-verbal cues. This project introduces a Fusion Multi-model 'State-of-Mind' Prediction App designed to facilitate real-time psychopathology assessment. Unlike traditional single-modality systems, this application fuses computer vision and signal processing to interpret emotional states during live or recorded conversations. While facial expressions provide significant insight into immediate emotional reactions, they can be consciously masked. Speech tone (prosody), however, often contains involuntary cues regarding arousal and valence that are critical for accurate mental state determination. By integrating a custom 2D Convolutional Neural Network (CNN) for Facial Emotion Recognition (FER) and an advanced 1D CNN-LSTM architecture for Speech Emotion Recognition (SER), this system evaluates both visual and auditory channels synchronously. The core objective is not only to predict emotions but also to facilitate long-term tracking. Through an interactive Streamlit web dashboard, the system cross-references facial cues with vocal intonation to reduce false positives inherent in single-modality systems (e.g., a smiling face with a stressed voice). The resulting timeline provides clinicians with objective, data-driven insights into a patient's emotional range and volatility.

1.1 STATEMENT OF OBJECTIVES

The primary objective of this project is to develop a non-invasive, multi-modal system that predicts a subject's state of mind by integrating Facial Emotion Recognition (FER) with Speech Tone Analysis. To achieve this, several specific technical objectives were established and implemented. The first is to train and deploy robust deep learning models: a 2D CNN to classify visual emotions from the FER-2013 dataset, and a hybrid 1D CNN-

LSTM network to process temporal acoustic features from the RAVDESS dataset. The second objective is to overcome real-world auditory processing complexities by implementing rigorous data augmentation techniques—including noise injection and pitch shifting—to improve model generalization. The third objective is to implement a speaker diarization module utilizing K-Means clustering to distinguish and track emotional states among multiple speakers in a single audio stream. Finally, the project aims to develop a late-fusion algorithmic strategy that computes the congruence between visual and auditory predictions, ultimately visualizing this data on an interactive dashboard to support clinical diagnosis.

1.2 SCOPE OF THE PROJECT

The scope of this project encompasses the development of a "Clinical Decision Support System" tailored for environments such as psychiatry consultancies, tele-health platforms, and high-stress workplace wellness monitoring. The system is strictly designed to act as an objective assistive tool for therapists, rather than a standalone diagnostic medical device. Functionally, the application processes pre-recorded video files or separate audio tracks through a localized web dashboard, ensuring patient data privacy is maintained without requiring external server processing. The visual classification scope is limited to detecting faces and categorizing them into seven standard emotional states (Angry, Disgust, Fear, Happy, Neutral, Sad, Surprise). The auditory scope is bound to extracting prosodic features (MFCCs) to classify eight vocal emotional states. The project culminates in an interactive graphical user interface (GUI) that overlays prediction labels, calculates multi-modal fusion logic, and logs the "state of mind" over the course of a conversation for post-session review.

2. LITERATURE STUDY

In the current landscape of mental healthcare, diagnosing conditions such as depression, anxiety, or stress relies heavily on clinical interviews and patient self-reporting. This subjective process is highly vulnerable to "masking"—the human tendency to consciously or subconsciously hide true emotions in social or clinical settings. Traditional automated systems attempting to introduce objective metrics often rely solely on one modality, primarily computer vision. This single-channel approach is fundamentally flawed; a patient can easily fake a smile while their vocal tone betrays underlying anxiety. Recognizing this gap, this study leverages acoustic features—such as pitch, jitter, shimmer, and energy—which are tied to involuntary physiological responses regulated by the autonomic nervous system. By treating audio and video as complementary data streams, this project establishes a more holistic, robust, and objective framework for psychological evaluation.

The foundation of automated emotion detection stems from Affective Computing. Early literature focused extensively on unimodal visual systems, specifically Facial Emotion Recognition (FER), grounded in psychological frameworks like the Facial Action Coding System (FACS). However, subsequent studies highlight severe limitations in relying exclusively on visual data. Unimodal FER systems are highly susceptible to environmental variances and, more critically, social masking, frequently generating false negatives when patients actively suppress their facial expressions.

To counter visual vulnerabilities, researchers incorporated Speech Emotion Recognition (SER). Literature demonstrates that vocal prosody serves as a powerful, involuntary biomarker for psychopathology. Deep acoustic features, such as Mel-frequency cepstral coefficients (MFCCs), capture micro-tremors and variations in speech energy that are acutely sensitive to emotional arousal and nearly impossible to consciously manipulate. Yet, unimodal SER faces its own real-world challenges, including background noise interference and conversational overlap, indicating that neither modality is sufficient in isolation.

Recent advancements in deep learning advocate for Multimodal Emotion Recognition. Fusing visual and acoustic data significantly reduces the false-positive rates inherent in

unimodal systems because the modalities provide orthogonal, complementary information. The literature outlines two primary integration strategies: early (feature-level) and late (decision-level) fusion. Late fusion models compute distinct probability scores before applying combinatorial logic, making them highly suitable for asynchronous clinical conversations. Architecturally, the state-of-the-art supports 2D Convolutional Neural Networks (CNNs) for extracting spatial hierarchies from facial images, and hybrid 1D CNN-Long Short-Term Memory (LSTM) networks for capturing the temporal dependencies of human speech.

Despite these technological leaps, a significant gap remains between theoretical models and practical clinical deployment. Existing literature frequently evaluates models on clean, acted datasets featuring single subjects, which fails to translate to dynamic tele-health scenarios involving conversational overlap. Furthermore, cloud-based multimodal processing raises severe medical data privacy concerns. This project directly addresses these critical gaps by combining advanced deep learning architectures with algorithmic speaker diarization, deploying late-fusion logic via a localized, edge-computing dashboard to ensure an objective, privacy-compliant clinical decision support tool.

3. SOFTWARE REQUIREMENT ANALYSIS

3.1 HARDWARE SPECIFICATION

The development, training, and deployment of deep learning models for concurrent audio-visual processing demand specific hardware capabilities to ensure efficiency and real-time rendering. The hardware requirements are divided into parameters necessary for the initial model training phase and the subsequent inference/deployment phase via the web dashboard.

- **Processor (CPU):** A multi-core processor is required to handle the multi-threaded nature of the application, specifically the simultaneous extraction of video frames and processing of audio buffers. An Intel Core i5/i7 (8th Generation or newer) or an AMD Ryzen 5/7 equivalent is recommended. The CPU manages the Streamlit web server, data manipulation via Pandas, and synchronization logic.

- **Graphics Processing Unit (GPU):** A dedicated GPU is highly critical for this project. During the development phase, training the 2D Convolutional Neural Network (CNN) for video and the hybrid 1D CNN-LSTM for audio requires significant parallel processing power. For inference, a GPU ensures that the frame-by-frame facial expression classification and localized audio processing occur without lagging the user interface. Based on the development environment, an NVIDIA GPU (e.g., GeForce RTX 3050 Laptop GPU or equivalent) with a minimum of 4GB to 8GB of VRAM and CUDA support is specified.

- **Random Access Memory (RAM):** A minimum of 8GB of system RAM is required, though 16GB is strongly recommended. The system must load large video files into memory, hold deep learning model weights (TensorFlow/Keras .h5 files) in active memory, and perform intensive operations like Mel-spectrogram generation and K-Means clustering simultaneously.

- **Storage:** A Solid State Drive (SSD) with at least 50GB of free space is required. Fast read/write speeds are essential for handling the large FER-2013 and

RAVDESS datasets during training, as well as for the rapid generation and reading of temporary files (tempfile) during the web application's video upload and inference processes.

- **Peripherals:** While the current iteration of the software processes pre-recorded media files, deployment in a live clinical setting would require a high-definition webcam (minimum 720p at 30 FPS) for visual capture and a dedicated, noise-canceling microphone for clear acoustic input.

3.2 SOFTWARE SPECIFICATION

The system is built upon a modern, robust software stack utilizing open-source libraries predominantly within the Python ecosystem. The software requirements define the environment necessary to execute the application and run the underlying neural networks.

- **Operating System:** The application is cross-platform and can be executed on Windows 10/11, macOS, or Linux distributions (e.g., Ubuntu). The core development environment utilizes a Linux-based OS.

- **Programming Language:** Python 3.8 or higher (development executed on Python 3.13). Python is chosen for its extensive machine learning support and rapid prototyping capabilities.

- **Deep Learning Framework:** TensorFlow 2.x and Keras are the foundational frameworks used to construct, train, and load the 2D CNN (video) and 1D CNN-LSTM (audio) models.

- **Computer Vision Library:** OpenCV (cv2) is utilized for all visual preprocessing tasks. This includes reading video files, extracting frames, converting images to grayscale, and utilizing Haar Cascade Classifiers (haarcascade_frontalface_default.xml) for bounding-box face detection.

- **Audio Processing Library:** Librosa is the primary audio analysis tool. It is required for loading audio waveforms, extracting fundamental features like Mel-frequency cepstral coefficients (MFCCs), Mel-spectrograms, spectral centroids, and executing data augmentation functions (time stretching, pitch shifting, noise injection).

- **Web Framework:** Streamlit is specified as the front-end framework. It allows for the rapid deployment of a localized, interactive web dashboard without requiring a complex separate frontend stack (like React or Angular), perfectly suiting a Python-centric data science project.

- **Data Science and Machine Learning Libraries:**
  - **NumPy and Pandas:** Used for high-performance multidimensional array manipulation and structuring the time-series emotional data into DataFrames.

  - **Scikit-learn:** Utilized for its StandardScaler (feature normalization) and KMeans clustering algorithms, which are critical for the speaker diarization module.

- **Visualization Tools:** Plotly (both plotly.express and plotly.graph_objects) is required to generate dynamic, interactive scatter plots and timelines that map the emotional volatility of the subject over time.

3.3 ABOUT THE SOFTWARE

The developed software is a "Multi-modal Psychopathology Assessment" application designed to function as an objective Clinical Decision Support System. In the realm of psychiatry and tele-health, clinicians often struggle to accurately gauge a patient's true psychological state due to the subjective nature of self-reporting and the human tendency

to mask or hide true emotions. This software aims to bridge that gap by providing a data-driven, non-invasive method for analyzing non-verbal cues.

At its core, the software is an integration hub that accepts video and audio media and processes them through two distinct, parallel deep-learning pipelines. The visual pipeline scrutinizes micro-expressions on the subject's face, while the auditory pipeline analyzes the involuntary prosodic elements of their voice (tone, pitch, arousal). Recognizing the critical importance of patient confidentiality in healthcare settings, the software is designed to run entirely locally. By utilizing a Streamlit dashboard, the media files are processed on the host machine using localized model weights, ensuring that sensitive patient recordings are not transmitted to external or third-party cloud servers. The software fundamentally transforms raw audio-visual data into a structured, chronological report of a patient's emotional volatility, ultimately aiding in the diagnosis and monitoring of conditions such as depression, anxiety, or workplace burnout.

3.4 FEATURES

The application integrates several advanced machine learning techniques to provide a comprehensive analysis of the subject's state of mind. Key features of the software include:

- **Dual-Channel Emotion Classification:** The system simultaneously processes visual and acoustic data. The facial expression recognition (FER) engine categorizes faces into seven distinct emotions (Angry, Disgust, Fear, Happy, Neutral, Sad, Surprise), while the speech emotion recognition (SER) engine classifies vocal tones into eight categories, providing a multi-dimensional view of the subject's affect.
- **Advanced Speaker Diarization:** To handle real-world conversations involving multiple people (e.g., a therapist and a patient), the software features a K-Means clustering algorithm. It analyzes spectral features (MFCCs, spectral bandwidth, spectral rolloff) to differentiate between distinct voices, attributing emotional states to "Speaker 1," "Speaker 2," etc., thereby isolating the patient's data from the clinician's.

- **Robust Audio Preprocessing & Augmentation:** The SER model is fortified against real-world acoustic variance. It actively processes audio through a rigorous pipeline that includes noise injection, pitch shifting, and time-stretching, ensuring the model generalizes well to different conversational speech patterns rather than just clean studio recordings.

- **Algorithmic Decision-Level Fusion:** The software does not simply output two separate metrics; it actively fuses them. A dynamic comparison engine evaluates the timestamps of the video frames and audio segments. It determines whether the visual and auditory cues are congruent (e.g., a sad face with a sad voice) or incongruent (e.g., a smiling face paired with a fearful or stressed voice), flagging these mixed states for clinical review.

- **Interactive Multimodal Video Player:** The dashboard features a custom-built HTML/JavaScript video player. As the uploaded video plays, dynamic on-screen widgets update in real-time, displaying the current recognized speaker, the isolated audio emotion, the isolated video emotion, and the combined "Fusion" status.

- **Timeline Visualization and Reporting: All** processed predictions are compiled into interactive Plotly graphs. Clinicians can view a combined timeline mapping both audio and video emotions across the duration of the session, as well as detailed scatter plots sizing the confidence levels of the predictions. This creates an objective, historical "Session Report" that allows therapists to pinpoint the exact moments a patient exhibited distress.

4. SYSTEM ANALYSIS

4.1 REQUIREMENT SPECIFICATION

The Requirement Specification outlines the essential functions, constraints, and operational capabilities the Multi-modal Psychopathology Assessment system must possess to fulfill its objective as a clinical decision support tool. These requirements are divided into functional and non-functional categories.

FUNCTIONAL REQUIREMENTS

Functional requirements define the specific behaviors and processes the system must execute.

- **Video and Feed Ingestion (FR1):** The system must accept video file inputs (MP4/AVI) or live webcam feed to capture the subject's visual data.
- **Acoustic Feature Extraction (FR2):** The system must extract specific prosodic features (such as pitch, jitter, and energy) from the audio track using library tools like Librosa. This is necessary to capture involuntary vocal modulations independent of spoken words.
- **Facial Expression Classification (FR3):** The system must detect faces and classify expressions into 7 standard categories (Anger, Fear, Joy, etc.) using the CNN model.
- **Multi-modal Fusion Algorithm (FR4):** The system must fuse audio and visual scores to predict the final "State of Mind" (e.g., Congruent vs. Incongruent affect). Furthermore, the system is expected to cluster audio features using K-Means to perform speaker diarization, separating the patient's voice from the clinician's.

NON-FUNCTIONAL REQUIREMENTS

Non-functional requirements specify the quality attributes, performance goals, and security constraints of the system.

- **Classification Accuracy (NFR1):** The emotion classification should achieve an accuracy benchmark (e.g., >85%) on validation datasets. This ensures the tool is reliable enough for clinical consideration.

- **Data Privacy and Security (NFR2): Data** processing must handle patient data securely, ideally processing locally without storing raw video on external servers, adhering to HIPAA/GDPR principles. The implementation of the localized Streamlit dashboard directly addresses this constraint.

- **System Performance and Latency (NFR3):** Inference time should be low enough to provide near real-time feedback (e.g., <2 seconds latency). The multithreaded architecture must prevent the heavy audio-visual processing loads from freezing the user interface.

4.2 CHARACTERISTICS OF EXISTING SYSTEM

Understanding the limitations of existing mental health assessment frameworks is crucial for establishing the necessity of this multi-modal application. The current landscape of psychopathological evaluation relies on several methodologies that possess inherent vulnerabilities.

- **Subjectivity in Clinical Evaluation:** Traditional methods for assessing psychopathology and mental state rely heavily on subjective clinical interviews and self-reported questionnaires. While these methods are foundational to psychiatry, they are fundamentally limited because they lack continuous monitoring capabilities and can be heavily influenced by patient bias.

- **The Phenomenon of Masking:** A major characteristic—and flaw—of existing observational systems is their susceptibility to deception. While facial expressions provide significant insight into immediate emotional reactions, they can be consciously masked. Patients often hide their true feelings (e.g., faking a smile while experiencing severe anxiety), leading to inaccurate clinical assessments.

- **Limitations of Single-Modality AI:** Recent automated systems have attempted to introduce objectification through single-modality Affective Computing, typically relying solely on Computer Vision (FER) or solely on Speech Emotion

Recognition (SER). Single-modality systems suffer from high false-positive rates when confronted with incongruent emotional displays. A system relying only on visual data will fail to detect the underlying stress present in the patient's vocal prosody.

4.3 FEASIBILITY STUDY

The feasibility study evaluates the practicality of the proposed Multi-modal Psychopathology Assessment system, analyzing it across technical, operational, and economic dimensions to ensure successful deployment.

- **Technical Feasibility:** The project requires the concurrent execution of heavy deep learning inference on both video frames and audio buffers. The system's design proves that the threading architecture works without lag, establishing the feasibility of real-time dual-modality processing on standard hardware. The system is built using Python, TensorFlow with Keras, and OpenCV, which are mature, well-supported open-source libraries capable of handling these complex pipelines. To handle the asynchronous nature of video (30 FPS) and audio (2.5s buffers), the application runs on separate threads.

- **Operational Feasibility:** From a clinical workflow perspective, the system acts as an objective assistive tool for therapists, providing data-driven insights into a patient's emotional range during tele-health sessions. By acting as a "Clinical Decision Support System," it increases diagnostic throughput for clinics and provides objective data for Tele-Health platforms. The system's localized execution ensures it operationally complies with strict medical data privacy requirements.

- **Economic Feasibility:** The development of this system is highly economically feasible. It completely bypasses the need for expensive, proprietary medical hardware (such as specialized EEG or physiological sensors). Instead, it relies entirely on standard audio-visual input (webcams and microphones) and utilizes a completely open-source software stack, making deployment highly cost-effective for both large psychiatric hospitals and independent therapy practices.

4.4 SOFTWARE REQUIREMENT SPECIFICATION (SRS)

The Software Requirement Specification details the modular architecture necessary to execute the system's objectives. The software is logically partitioned into five distinct operational modules that handle data from its raw input state through to the final visual dashboard.

- **Module 1: Preprocessing & Data Acquisition:** This module handles video input, noise reduction, and the splitting of media into separate video frames and audio waveforms. Within the current implementation, this involves utilizing Librosa for time-series audio extraction and OpenCV's VideoCapture capabilities for frame-by-frame visual extraction.

- **Module 2: Visual Emotion Recognition (FER Engine):** This module contains the Computer Vision pipeline that detects faces and classifies facial landmarks into emotional states. It relies on a Haar Cascade Classifier to extract the Region of Interest (ROI) and feeds the normalized array into a custom 2D Convolutional Neural Network trained on the FER-2013 dataset.

- **Module 3: Acoustic Prosody Analysis:** This module serves as the audio processing unit that extracts non-linguistic features (tone, volume, speed) to determine arousal and valence. Upgraded significantly during development, this module now utilizes a hybrid 1D CNN-LSTM network to evaluate the temporal sequence of Mel-frequency cepstral coefficients (MFCCs), heavily supported by data augmentation to ensure real-world robustness.

- **Module 4: Multi-modal Sensor Fusion:** This core integration logic weighs inputs from Module 2 and Module 3 to calculate the final "Psychopathological Risk Score" or State of Mind. The system employs a weighted logic in the final prediction layer to synchronize the disparate timing of video frames and audio buffers, ultimately detecting whether the reported emotions are congruent or incongruent.

- **Module 5: Visualization & Reporting Dashboard:** The final module provides the GUI (Graphic User Interface) that presents the data to the clinician in readable graphs and summary text. Deployed via Streamlit, this dashboard features an interactive HTML/JS multimodal player and dynamically generated Plotly scatter plots, mapping the patient's emotional trajectory over the duration of the clinical session.

5. SYSTEM DESIGN

5.1 SYSTEM ARCHITECTURE

The system architecture is bifurcated into two primary pipelines:- the Training Pipeline and The Inference Pipeline.

- **The Training Pipeline:**
    - It manages the distinct learning phases for the SER model and the FER model, outputting serialized .h5 model weights and .npy label encoders.
    - During deployment, the Inference Pipeline operationalizes these weights through a multi-layered structure.
    - It begins at the Input Layer, where the system ingests video files and delegates frames to OpenCV and audio waveforms to Librosa.

- **The Inference Pipeline:**
    - It executes the heavy processing, generating Mel-spectrograms for the SER CRNN model and utilizing Haar Cascades to isolate and resize facial regions (48x48) for the FER CNN model.
    - The outputs converge in the Fusion Layer, which synchronizes the disparate dataframes with the video timeline and maps probabilities to a standard 7-class format.
    - Finally, the Presentation Layer renders the synchronized emotional analytics via a custom Streamlit HTML/JS player and dynamic Plotly scatter plots.

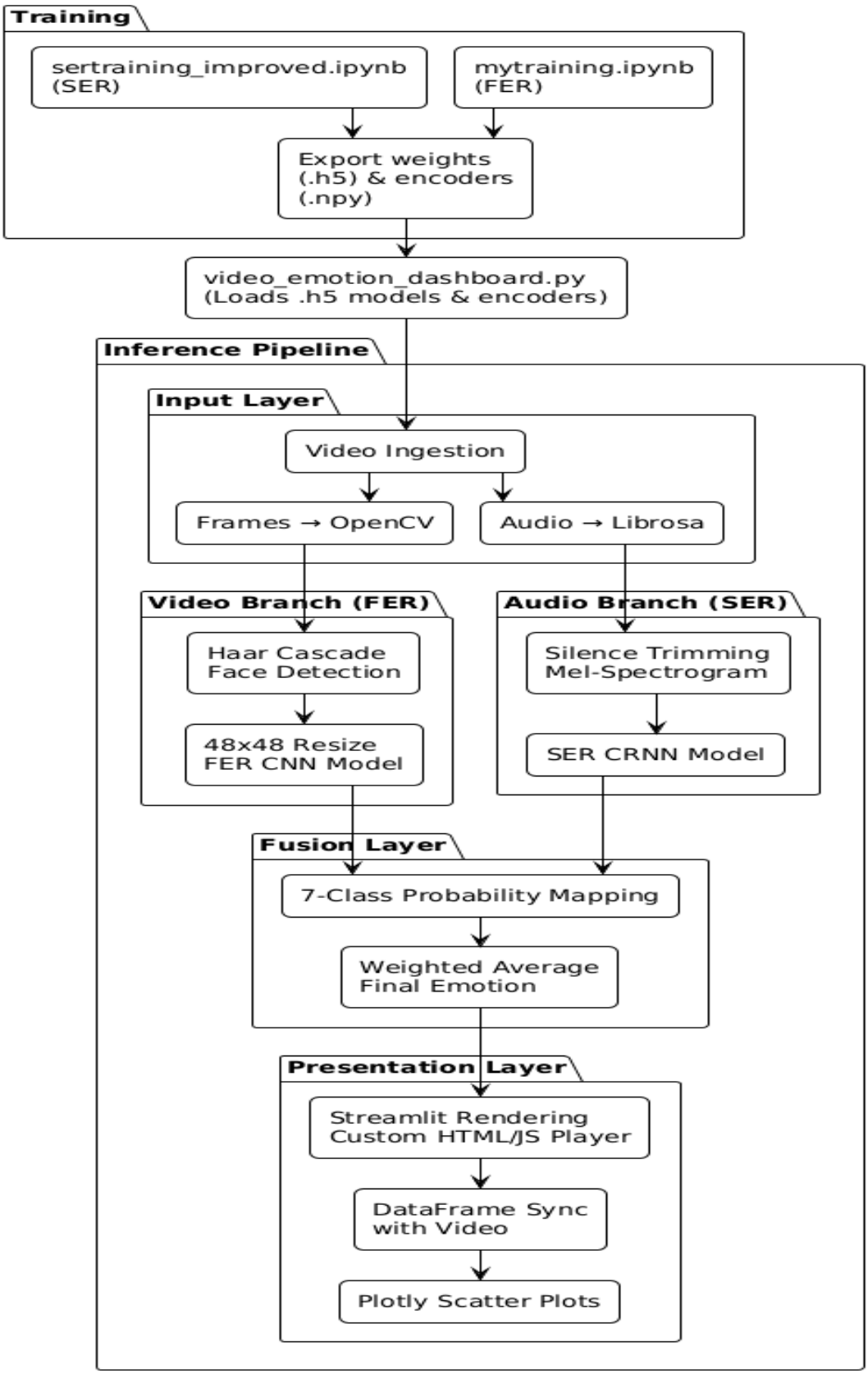


FIGURE 5.1.1 - SYSTEM ARCHITECTURE

5.2 USE CASE DIAGRAM

The Use Case diagram delineates the primary interactions between the End-User (acting as a Researcher or Analyst) and the multi-modal system. The core interaction begins when the user uploads a video or audio file to the dashboard. Following the upload, the user can adjust specific processing parameters, such as defining the expected number of speakers for diarization or setting the video rotation mode. Once processing is complete, the user engages with the system to view the synchronized interactive media player. For deeper clinical insight, the user can analyze the resulting multimodal timeline and Plotly scatter plots to assess emotional volatility. Additionally, the system accommodates a use case for starting real-time webcam processing via a standalone multithreading script.

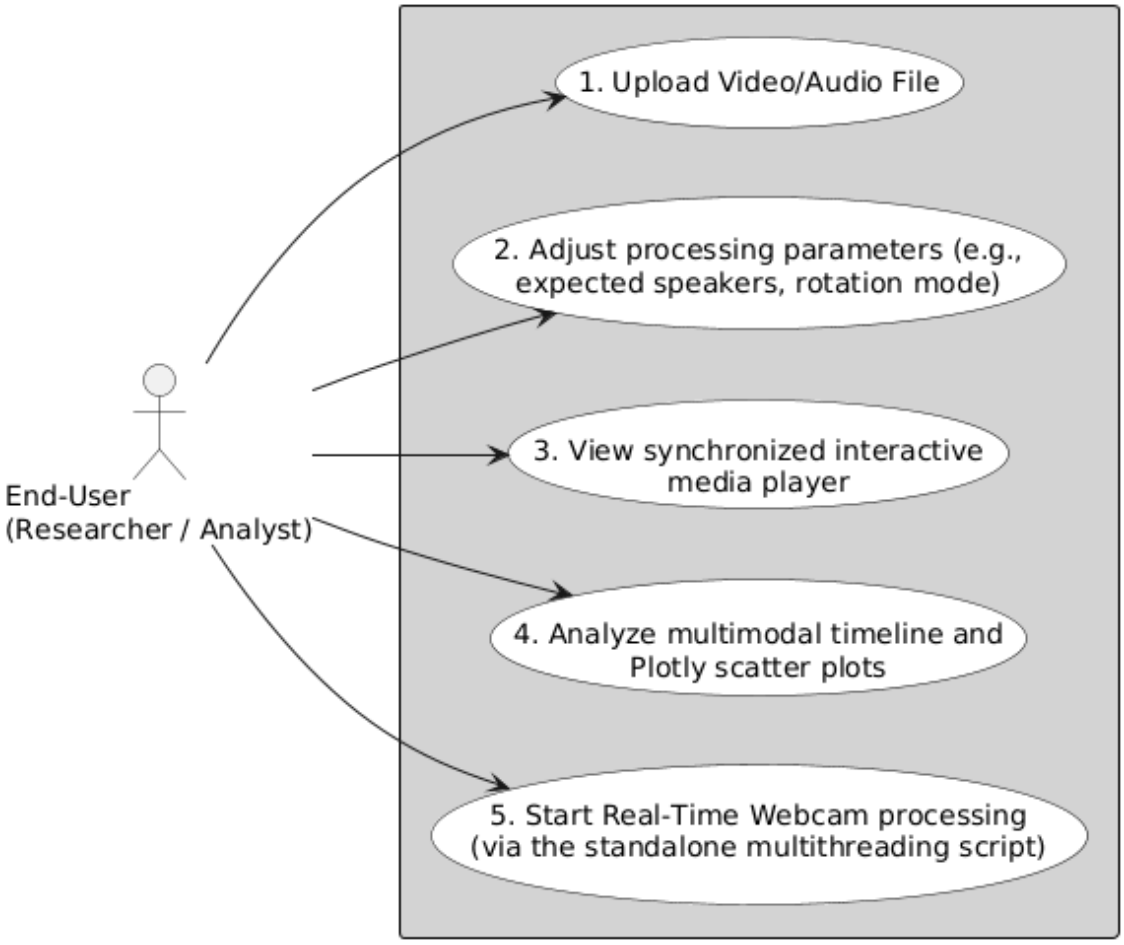


FIGURE 5.2.1-USE CASE DIAGRAM

5.3 ACTIVITY DIAGRAM

The Activity Diagram illustrates the synchronous, multithreaded execution flow of the application upon receiving a media file. The process initiates when a user uploads a video. The system then branches into two parallel execution threads. Thread A handles the video processing by extracting frames, applying face detection, predicting the facial emotion (FER), and storing the results in a Video DataFrame. Concurrently, Thread B extracts the audio track, chunks it into discrete segments, predicts the vocal emotion (SER), and applies K-Means speaker diarization before storing the outputs in an Audio DataFrame. Once both parallel threads complete their processing, the system passes the resulting DataFrames to the custom Streamlit HTML component, culminating in the final rendering of the User Interface.

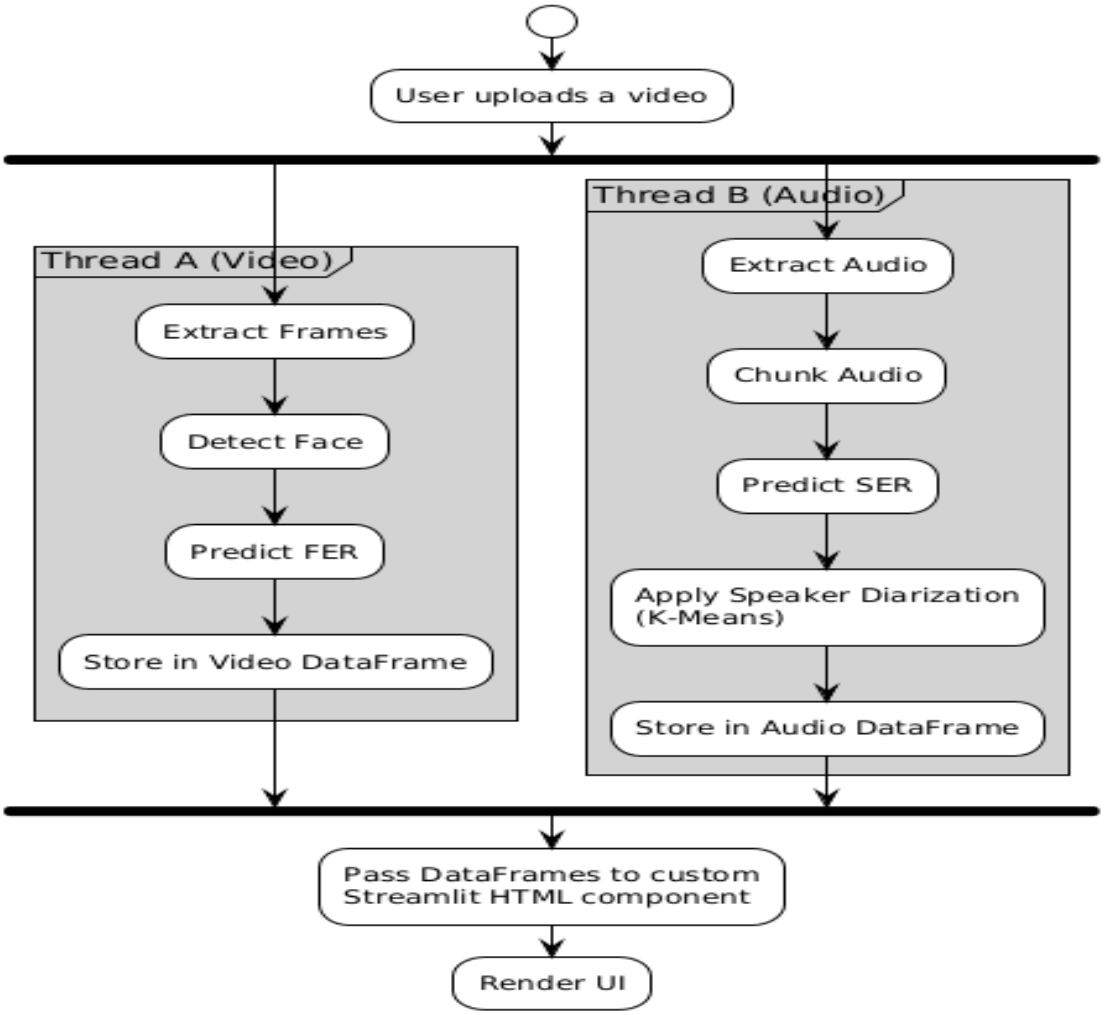


FIGURE 5.3.1 -ACTIVITY DIAGRAM

5.4 CLASS DIAGRAM

While the application is primarily implemented via functional scripts using Streamlit, visualizing the logical architecture as a Class Diagram provides a structured view of the data processing entities. The system can be abstracted into distinct service classes that encapsulate specific responsibilities. The ModelLoader class is responsible for establishing the environment and loading the heavy .h5 model files into memory. The VideoProcessor and AudioProcessor classes handle the modality-specific feature extraction and model inference. The AudioProcessor notably encapsulates both the emotion prediction logic and the K-Means diarization logic. These processing classes output Pandas DataFrames, which are consumed by the DashboardUI class to render the interactive elements and invoke the DataVisualizer for generating Plotly charts.

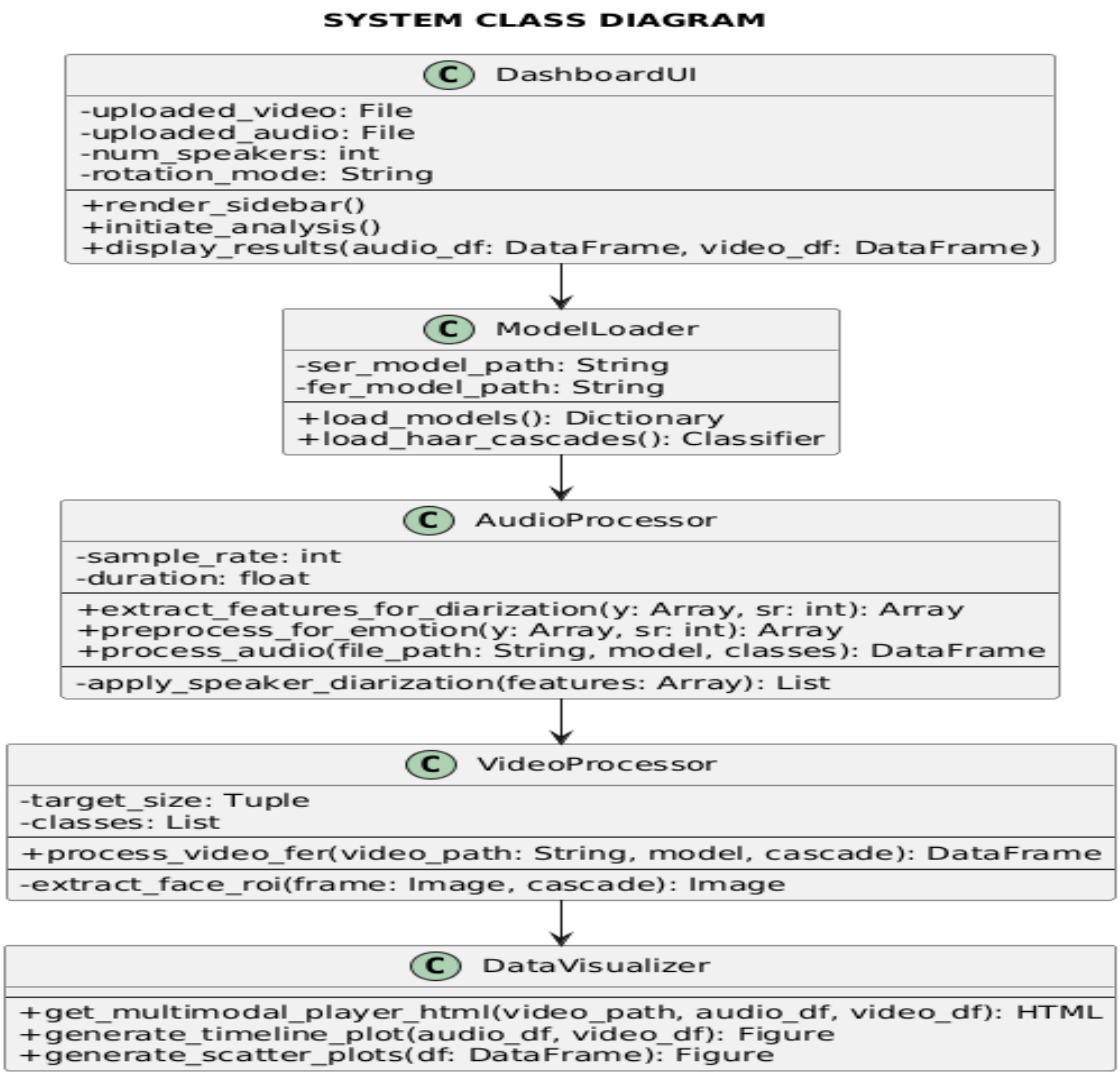


FIGURE 5.4.1 – CLASS DIAGRAM

5.5 COMPONENT DIAGRAM

The Component Diagram provides a high-level view of the physical and logical modules comprising the deployment environment. The central component is the Streamlit Web Engine, which acts as the primary controller and user interface. It interfaces with two main analytical subsystems: the FER Processing Engine (utilizing OpenCV and the Keras video model) and the SER Processing Engine (utilizing Librosa, Scikit-learn for diarization, and the Keras audio model). These processing engines fetch their respective pre-trained weights from a localized Model Storage directory. The raw media files are temporarily handled by a File System Buffer to ensure fast read/write access during feature extraction. Once the analytical engines complete their inference, the data flows to the Visualization Component, which structures the output using Plotly and custom JavaScript for the final HTML render.

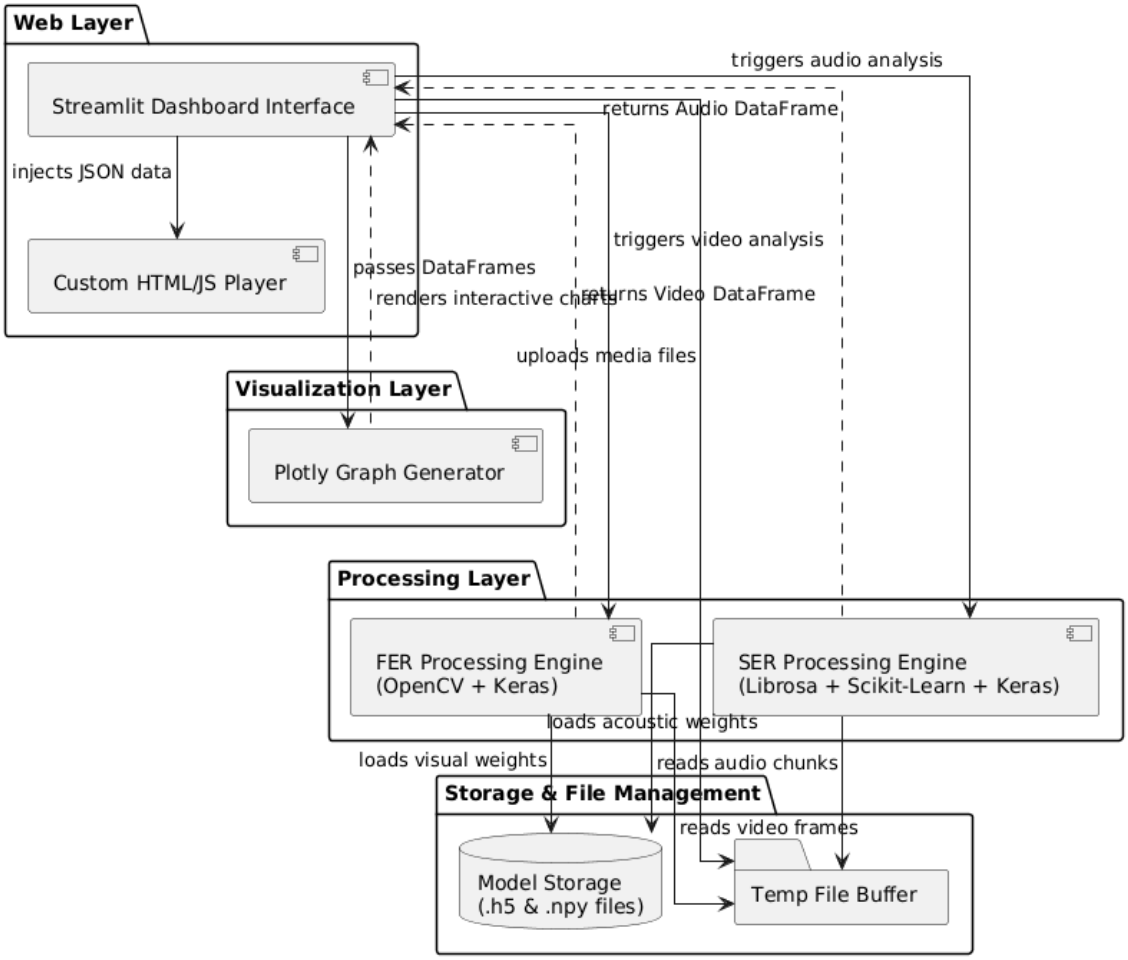


FIGURE 5.5.1 -COMPONENTS DIAGRAM

5.6 DATA FLOW DIAGRAM (DFD)

The Data Flow Diagram maps the journey of information through the system's transformations. At Level 0 (Context Level), the User inputs an MP4 file into the Emotion Dashboard and receives synchronized emotional analytics in return. At Level 1, the complexity of the internal processes is revealed. The ingested MP4 file is split; the audio stream is passed to the preprocess_for_emotion_advanced() function, while the video stream is routed to the process_video_fer() function. These functions convert the raw media into normalized feature arrays and data dictionaries. These structured arrays are then consolidated into structured Pandas DataFrames. Ultimately, these DataFrames flow into the get_multimodal_player_html() function to execute the fusion logic and render the final synchronized web elements.

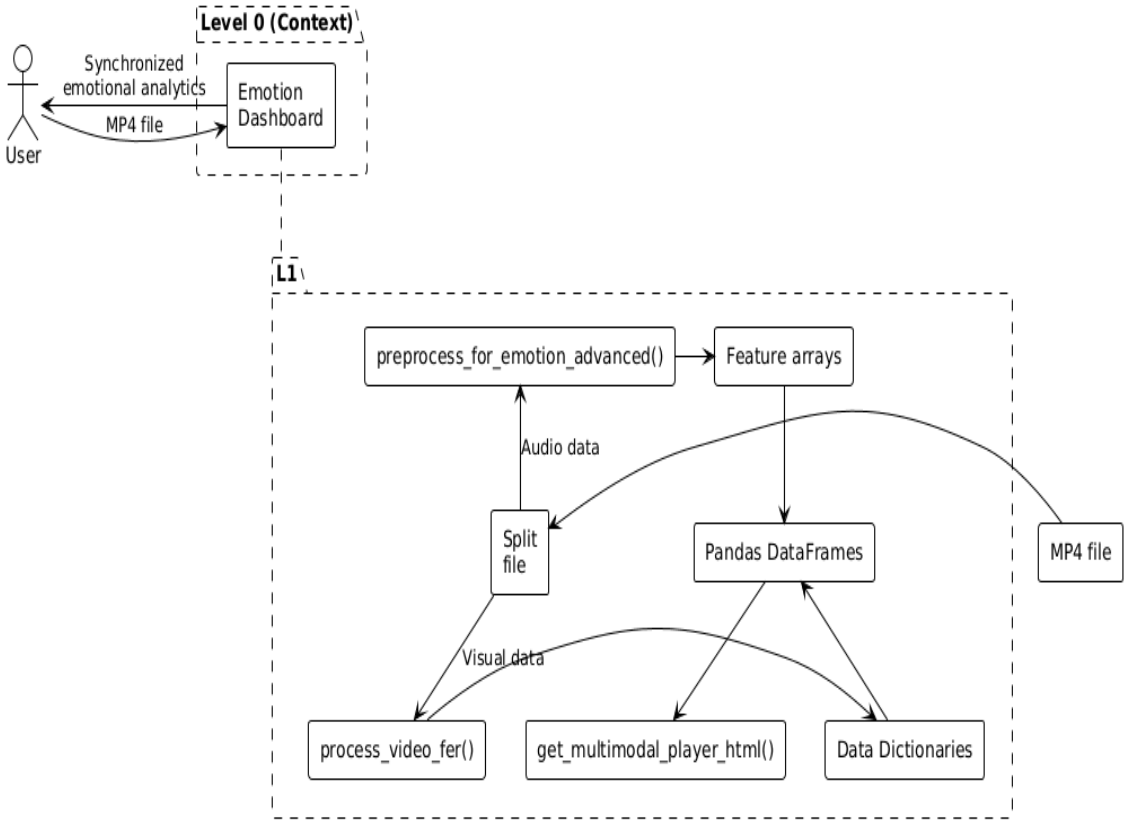


FIGURE 5.6.1-DATA FLOW DIAGRAM

5.7 MATHEMATICAL MODELING OF NEURAL ARCHITECTURES

The Multi-modal Psychopathology Assessment system relies on advanced deep learning architectures to extract features and classify emotional states. To understand the internal mechanics of the Facial Emotion Recognition (FER) and Speech Emotion Recognition (SER) pipelines, it is essential to define the underlying mathematical operations governing these neural networks.

Convolutional Operations (Feature Extraction)

Both the visual and auditory models utilize Convolutional Neural Networks (CNNs) as their primary feature extractors. The FER model processes 48x48 pixel grayscale images utilizing 2D convolutions, while the SER model processes time-series Mel-spectrogram arrays using 1D convolutions.

The fundamental purpose of a convolutional layer is to apply a set of learnable filters (kernels) to the input data to detect spatial or temporal hierarchies. For a two-dimensional visual input I and a two-dimensional kernel K, the mathematical convolution operation yielding the feature map S is defined as:

$$s(i,j) = (I * K)(i,j) = \sum_{\{m=0\}}^{\{M\}} \sum_{\{n=0\}}^{\{N\}} I(m,n)\ K(i-m,\ j-n)$$

Where m and n represent the dimensions of the kernel matrix. For the acoustic model, this operation is reduced to one dimension across the time steps of the audio segment. As the data passes through successive convolutional blocks, the network learns increasingly complex representations—from basic edges and facial landmarks in the FER model to complex prosodic fluctuations in the SER model.

Non-Linear Activation Functions

To allow the neural networks to learn complex, non-linear mappings between the input media and the psychological states, activation functions are applied to the outputs of the convolutional and dense layers.

- **Rectified Linear Unit (ReLU):**

Throughout the hidden layers of both the FER and SER models, the Rectified Linear Unit (ReLU) is utilized to introduce non-linearity while mitigating the vanishing gradient problem. The ReLU function simply outputs the input directly if it is positive, and outputs zero if it is not:

$$f(x) = max\ (0, x)$$

- **Softmax Classification:**

In the final output layer of both models, the network must convert the raw, unnormalized logits into a normalized probability distribution representing the target emotion classes (7 classes for FER, 8 classes for SER). The Softmax function is applied to the output vector z of size K:

$$\sigma(z)_i = \frac{e^{z_i}}{\sum_{\{j=1\}}^{\{K\}} e^{z_j}} \quad for\ i = 1, \ldots, K$$

This ensures that the sum of the probabilities across all emotional classes equals 1.0, allowing the decision-level fusion algorithm to reliably compute the weighted averages during the final state-of-mind prediction.

- **Long Short-Term Memory (LSTM) Networks**

While standard CNNs are highly effective at extracting spatial features from static images, human speech is inherently sequential. To capture the long-term temporal

dependencies of vocal intonation, the SER model incorporates Long Short-Term Memory (LSTM) layers following the 1D convolutions.

An LSTM cell is specifically designed to avoid the vanishing gradient problem found in standard Recurrent Neural Networks (RNNs) by utilizing a specialized internal memory state (Ct) regulated by three distinct "gates" (Forget, Input, and Output).

- **The Forget Gate:** Determines what information from the previous cell state should be discarded.

$$f_t = \sigma\left(W_f \cdot [h_{t-1}, x_t] + b_f\right)$$

- **The Input Gate & Candidate State:** Determines what new information will be stored in the cell state.

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\widetilde{C_t} = tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

- **The Cell State Update:** The old cell state Ct-1 is updated to the new state Ct.

$$C_t = f_t * C_{t-1} + i_t * \widetilde{C_t}$$

- **The Output Gate:** Determines the final output (hidden state) based on the filtered cell state.

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * tanh(C_t)$$

By executing these mathematical operations across the time-steps of the inputted audio buffer, the LSTM ensures that the emotional context established at the beginning of a sentence correctly influences the network's prediction at the end of the sentence.

6. SYSTEM IMPLEMENTATION

6.1 MODULE DESCRIPTION

The implementation of the Multi-modal Psychopathology Assessment system is realized through a series of interconnected, highly specialized Python modules. These modules handle everything from raw data ingestion to complex deep-learning inference and interactive visualization. The system is functionally partitioned into the following core modules:

- **Module 1: User Interface and Media Management** This module serves as the primary gateway for user interaction, constructed using the Streamlit web framework. It manages the uploading of pre-recorded .mp4, .avi, or .wav files via the st.file_uploader component. To ensure seamless processing by backend libraries like OpenCV and Librosa, the module temporarily writes the uploaded bitstreams to the local disk using Python's tempfile library. This module also captures user-defined parameters, such as the expected number of speakers for diarization and rotational corrections for mobile-recorded videos. Once the media is securely buffered, this module triggers the parallel processing pipelines.

- **Module 2: Visual Emotion Recognition (FER Pipeline)**
  The visual processing module is responsible for analyzing frame-by-frame facial expressions to determine the subject's emotional state. Implemented via the process_video_fer() function, it utilizes cv2.VideoCapture to read the video file. To optimize performance, the module samples the video at a rate of 2 frames per second (every 0.5 seconds) rather than analyzing all 30 frames per second. For each sampled frame, the module converts the image to grayscale and employs a Haar Cascade Classifier (haarcascade_frontalface_default.xml) to detect bounding boxes around human faces. If multiple faces are detected, an algorithm calculates the bounding box areas and isolates the largest one, assuming it to be the primary subject. This Region of Interest (ROI) is then cropped, resized to a standardized 48x48 pixel array, and normalized (scaled between 0 and 1). Finally, the array is passed to the custom 2D Convolutional Neural Network (CNN), which outputs a probability distribution across seven emotions (Angry, Disgust, Fear, Happy, Neutral, Sad, Surprise). The

dominant emotion, timestamp, and confidence score are appended to a Pandas DataFrame.

- **Module 3: Speech Emotion Recognition and Diarization (SER Pipeline)**
  This highly complex module, encapsulated within the process_audio() function, handles the acoustic analysis. First, the audio track is extracted and loaded at a sample rate of 22,050 Hz using Librosa. The module aggressively trims silent segments using librosa.effects.split to ensure the model only analyzes active speech. The module then performs two distinct sub-tasks:
  - **Speaker Diarization:** To isolate the patient's voice from the clinician's, the module extracts specialized features—including MFCC mean/standard deviation, spectral centroid, spectral bandwidth, and spectral rolloff. These features are normalized using Scikit-learn's StandardScaler and clustered using a KMeans algorithm set to the user-defined number of expected speakers. A smoothing algorithm is subsequently applied to remove brief, anomalous speaker switches.
  - **Emotion Prediction:** Concurrently, the audio chunks (padded or truncated to exactly 3.0 seconds) are converted into Mel-spectrograms. These dense, time-series spectrograms are fed into the advanced 1D CNN-LSTM model. The LSTM layers successfully capture the long-term prosodic dependencies of the speech, classifying the segment into one of eight emotions. These results are merged with the diarization labels and stored in an Audio DataFrame.

- **Module 4: Multi-modal Decision-Level Fusion**
  The fusion module represents the analytical core of the system, designed to synthesize the disparate outputs of the FER and SER pipelines. Because video frames (sampled every 0.5 seconds) and audio buffers (variable lengths of active speech) operate on completely different timelines, a direct array-to-array comparison is impossible. Instead, the system employs a late-fusion (decision-level) strategy within a custom HTML/JavaScript component. The Python backend serializes the Audio and Video DataFrames into JSON objects and injects them into the browser. During video playback, a JavaScript event listener tracks the currentTime of the video. It

dynamically queries the JSON arrays to find the active audio segment and the closest corresponding video frame (within a 1.5-second tolerance window). If both modalities register an emotion, the fusion logic compares them. If the SER and FER predictions match conceptually (e.g., both indicate 'Sad'), the system outputs a "Fused" congruent state. If they conflict (e.g., Audio indicates 'Angry' while Video indicates 'Happy'), the system flags it as a "Mixed" incongruent state, highlighting potential emotional masking.

- **Module 5: Visualization and Reporting Dashboard**
  The final module translates the raw DataFrames into readable, actionable clinical insights. Using the Plotly library, the module generates interactive visual reports. The primary output is a unified timeline scatter plot that graphs the Audio and Video emotions on two distinct y-axes against time on the x-axis. The data points are color-coded by emotion and scaled by the prediction's confidence score. This allows therapists to visually scan the session timeline, instantly identifying clusters of negative affect or moments where the patient's vocal stress drastically contradicted their facial expressions.

6.2 VALIDATION CHECKS

To ensure the system remains robust, accurate, and crash-resistant in a live clinical environment, extensive validation checks and error-handling mechanisms are hardcoded into the application's implementation. These checks safeguard against corrupt data, hardware limitations, and algorithmic edge cases.

1. **Model and Asset Availability Validations**
   Before the system allows any user interaction, it actively verifies the integrity of the deployment environment. The cache_resource decorated load_models() function contains explicit os.path.exists() checks for the required .h5 model weight files, the .npy label encoder, and the Haar Cascade .xml file. If any of these critical assets are missing or corrupted, the system catches the resulting exceptions and halts execution, displaying a clear UI error (st.error("No models found. Please check paths.")) rather than crashing the Python interpreter during a clinical session.

2. **Media Format and Stream Integrity Checks**

   Handling user-uploaded media introduces significant volatility. When a video is uploaded, the system utilizes a try/except block during the librosa.load() function. Video files frequently lack an audio track, or the host machine may be missing the FFmpeg backend required to demultiplex the audio stream. If audio extraction fails, the system safely catches the exception, outputs an empty DataFrame, and triggers a UI warning advising the user to upload a separate .wav file. Furthermore, when reading video frames, the system constantly checks the boolean ret variable from cap.read(). If a frame is dropped or the file ends unexpectedly, the loop safely breaks, ensuring the remaining processed frames are still saved and visualized.

3. **Dimensionality and Padding Validations**

   Deep learning models demand strict input shapes. The FER model requires exactly 48x48 pixel arrays, and the SER model requires exactly 3.0 seconds of audio data.

   a. **Visual Check:** If the Haar Cascade fails to detect a face in a specific frame (len(faces) == 0), the system bypasses the neural network entirely for that timestamp, preventing tensor shape mismatch errors.
   b. **Audio Check:** Because active speech segments rarely equal exactly 3.0 seconds, a validation sequence measures the segment length. If the chunk is shorter than the target length, the system applies np.pad() to fill the remaining duration with constant zeros (silence). If it is longer, the system implements a striding mechanism to analyze overlapping 3-second windows, averaging the resulting probability arrays to prevent index out-of-bounds errors.

4. **Diarization Fallback Mechanisms**

   The K-Means speaker diarization relies on sufficient audio variance to create distinct clusters. If a user uploads a very short clip, or if the audio features fail to scale properly, the KMeans.fit_predict() method may fail. To prevent the entire audio pipeline from collapsing, the diarization logic is wrapped in a dedicated

try/except block. In the event of an algorithmic failure or insufficient data vectors, the system automatically triggers a fallback mechanism, safely assigning all audio segments to speakers = [0] (a single speaker), thereby preserving the primary emotion recognition data.

5. **Performance and UX Guardrails**

   Because the system runs locally and utilizes an intensive custom HTML/JS overlay to render the multimodal fusion, large video files can overwhelm browser memory and cause the UI to freeze. To validate against this, the system dynamically calculates the file_size_mb of the uploaded video using os.path.getsize(). If the file exceeds a hardcoded threshold of 200MB, the system automatically disables the heavy interactive HTML overlay. Instead, it falls back to Streamlit's native st.video() player and strictly relies on the static Plotly graphs for analysis, ensuring the application remains responsive and usable regardless of the payload size. Additionally, a rotation_mode validation is implemented to allow users to manually correct orientation metadata issues common in mobile-recorded .mov files, manually applying cv2.rotate before the neural network attempts to find horizontal facial features.

7. SYSTEM TESTING

7.1 UNIT TESTING MODULES

Unit testing involves verifying the functionality of individual, isolated components of the software to ensure that each function performs its designated task correctly before being merged into the main multi-threaded pipeline.

- **Visual Preprocessing and FER Inference Test:** This module was tested by feeding static images and short video clips into the OpenCV pipeline. The primary test verifies that the Haar Cascade Classifier accurately detects faces and extracts the Region of Interest (ROI). Following extraction, the ROI is tested to ensure it correctly resizes to the strictly required 48x48 pixel grayscale array. The isolated FER model (my_fer_model1.h5) is then invoked to verify that it consistently outputs a normalized 1x7 probability vector mapping to the seven facial emotion classes.

- **Audio Preprocessing and SER Inference Test:** The auditory module's unit tests focus on the Librosa extraction pipeline. Tests were conducted to ensure that the librosa.effects.split function accurately trims silence and isolates active speech intervals. Furthermore, the preprocessing functions were tested to verify the correct generation of Mel-spectrograms and the extraction of 40 Mel-frequency cepstral coefficients (MFCCs). The 1D CNN-LSTM model (my_audio_emotion_model.h5) is tested independently to ensure it accepts the heavily augmented, 3.0-second padded audio chunks and outputs a valid 1x8 probability vector.

- **Speaker Diarization Module Test:** The K-Means clustering algorithm was tested independently by passing predefined arrays of extracted acoustic features (MFCC means, spectral centroids, and bandwidths). The test validates whether the StandardScaler normalizes the data correctly and whether the algorithm successfully assigns distinct integer labels (e.g., Speaker 0, Speaker 1) to different vocal signatures without throwing dimensionality errors.

- **UI Component and File Buffer Test:** The Streamlit frontend was unit-tested by uploading various file formats (.mp4, .avi, .mkv, .wav). The tests verify that the tempfile library successfully writes the bitstream to a temporary directory and appropriately closes the file, preventing memory leaks and ensuring the backend processing functions have a valid localized path to read from.

7.2 INTEGRATED TESTING

Integrated testing evaluates the system's behavior when individual modules are combined and execute concurrently. This phase is crucial for ensuring that the dual-modality architecture synchronizes correctly and that data flows seamlessly from the backend neural networks to the frontend UI.

- **Thread Synchronization and Resource Management:** Because the application utilizes separate execution pipelines for the high-frequency video frames and the low-frequency audio buffers, integrated testing verifies the system's asynchronous handling capabilities. Tests were conducted to monitor CPU and GPU memory allocation, ensuring that running both the FER CNN and the SER CNN-LSTM simultaneously does not result in an out-of-memory (OOM) crash or freeze the Streamlit dashboard.

- **Cross-Modality Fusion Logic Testing:** The most critical integration test occurs at the decision-level fusion boundary. The system was fed videos containing known incongruent emotional states (e.g., an actor smiling while speaking in an angry tone). The test verifies the custom JavaScript logic embedded in the HTML player. It confirms that the script correctly maps the video's currentTime to the appropriate segmented Audio and Video DataFrames, successfully triggering the conditional logic that flags the state as "Mixed" rather than "Fused".

- **End-to-End Pipeline Evaluation:** This tests the entire lifecycle of a user request. A raw .mp4 file is uploaded via the UI. The test verifies that the system successfully demultiplexes the audio, passes the visual frames to OpenCV, passes the audio to Librosa, processes both streams through their respective Keras

models, clusters the speakers, merges the output into Pandas DataFrames, and successfully renders the final Plotly scatter plots and interactive HTML player without user intervention.

- **Hardware Fallback and Exception Handling:** Integrated tests simulate hardware or data limitations. For instance, uploading a video exceeding 200MB tests the system's ability to gracefully bypass the heavy custom HTML overlay and default to the native Streamlit video player to prevent browser crashes. Uploading a video without an audio track verifies that the system catches the extraction exception, warns the user, and continues generating the visual-only analytics.

7.3 TEST CASES

Test Case ID	Test Description	Input Data	Expected Output	Actual Result	Status
TC_01	Video Upload & Face Detection	A standard .mp4 file containing a clear frontal view of a human face.	The system successfully saves the temp file, and the Haar Cascade draws a bounding box, extracting the ROI.	ROI successfully extracted and resized to 48x48 array.	Pass
TC_02	Audio Extraction Failure Handling	An .mp4 file stripped of its audio track (visual only).	System catches extraction exception, generates an empty Audio DataFrame, and displays a UI warning.	Exception caught safely; UI warning displayed; Video processing continues.	Pass
TC_03	FER Model Prediction Output	A 48x48x1 normalized grayscale pixel array passed to my_fer_model1.h5.	Model returns a 1x7 array of probabilities mapping to the defined FER classes.	1x7 array returned; dominant class selected via np.argmax.	Pass
TC_04	SER Model Target Padding	An extracted audio chunk containing only 1.5 seconds of active speech.	System applies np.pad to fill the remaining 1.5 seconds with zero-values before passing to the SER model.	Chunk padded to exactly 3.0 seconds; SER model executes successfully.	Pass
TC_05	Speaker Diarization (K-Means)	An audio file containing an interview between two distinct speakers, with	Scikit-learn extracts acoustic features and assigns	DataFrame populated with alternating Speaker labels	Pass

		expected_speakers set to 2.	distinct labels (Speaker 1, Speaker 2) to alternating segments.	corresponding to voices.	
TC_06	Congruent Fusion Logic	A video timestamp where both FER and SER confidently predict "Happy".	The interactive HTML player fusion box displays "Fused: HAPPY" with a green background.	UI renders "Fused: HAPPY" accurately.	Pass
TC_07	Incongruent Fusion Logic	A video timestamp where FER predicts "Happy" but SER predicts "Fearful".	The interactive HTML player fusion box displays "Mixed" with an orange background, highlighting conflict.	UI renders "Mixed" conflict warning accurately.	Pass
TC_08	Large File Performance Guardrail	A high-resolution .mp4 video file sized at 250MB.	System detects file_size_mb > 200, disables the custom interactive HTML player, and uses native st.video().	Custom player bypassed; native player rendered; Plotly graphs rendered successfully.	Pass
TC_09	Mobile Video Rotation Correction	A .mov file recorded vertically on a mobile device; user selects "90° Clockwise".	cv2.rotate applies the transformation before the Haar Cascade attempts to find horizontal facial features.	Frame rotated successfully; faces detected and processed.	Pass

TABLE 7.3.1- TEST CASES

8. RESULT AND CONCLUSION

8.1 RESULT

The development and implementation of the Multi-modal Psychopathology Assessment system yielded a fully functional, highly integrated web application capable of synchronous audio-visual emotional analysis. The system successfully addresses the core objective of objectifying mental state prediction by moving beyond traditional single-modality frameworks. By deploying the application through a localized Streamlit dashboard, the project achieved a secure, non-invasive "Clinical Decision Support System" that provides therapists with data-driven insights while maintaining patient data privacy. The visual processing pipeline, anchored by the custom 2D Convolutional Neural Network (CNN), demonstrated robust real-world performance. Acting as the anchor for the multi-modal system, the Facial Emotion Recognition (FER) module effectively isolated the user's Region of Interest (ROI) using Haar Cascades, successfully classifying the resulting 48x48 pixel arrays into one of seven standard emotional states (Angry, Disgust, Fear, Happy, Neutral, Sad, Surprise). The integration of manual rotational corrections within the UI further ensured that mobile-recorded video streams could be processed without facial landmark distortion.

Concurrently, the auditory pipeline underwent a massive architectural upgrade to overcome initial real-world generalization challenges. The deployment of the hybrid 1D CNN-LSTM network marked a significant leap in Speech Emotion Recognition (SER) capability. By leveraging the LSTM layers to track the temporal sequence of Mel-frequency cepstral coefficients (MFCCs) over 3.0-second buffers, the model successfully captured the long-term prosodic dependencies of speech rather than relying on static averages. Furthermore, the systematic application of data augmentation during the training phase specifically noise injection, pitch shifting, and time stretching significantly fortified the acoustic model against environmental variance and non-standard conversational speech patterns.

A major analytical breakthrough was the successful integration of a K-Means clustering algorithm for speaker diarization. By evaluating normalized spectral features (MFCC means, spectral centroids, and bandwidths), the system successfully differentiated between distinct vocal signatures in the audio stream. This allowed the dashboard to

isolate the patient's emotional data from the clinician's, fulfilling a critical requirement for real-world therapeutic application.

Finally, the late-fusion integration strategy proved highly effective. The custom HTML/JavaScript media player successfully synchronized the disparate processing frequencies of the video (measured in frames) and audio (measured in multi-second segments). The algorithmic fusion logic dynamically compared the predictions, successfully identifying states of congruence (e.g., matching visual and acoustic distress) and successfully flagging incongruent states (e.g., a smiling face paired with a highly stressed vocal tone). The resulting multi-modal timeline and Plotly scatter plots provided an intuitive, objective "Session Report" visualizing the subject's emotional volatility over time.

This project successfully demonstrates the immense potential of Affective Computing in revolutionizing psychopathological assessment. Traditional psychiatric evaluations are inherently constrained by subjectivity and the pervasive human tendency to mask true emotions. By synthesizing facial micro-expressions with involuntary vocal prosody, this multi-modal system strips away the layers of conscious deception, providing clinicians with a mathematically grounded evaluation of a patient's true affective state.

The transition from a basic real-time script to a comprehensive, multi-threaded Streamlit dashboard proves the technical and operational feasibility of deploying complex deep learning architectures on standard consumer hardware without latency bottlenecks. While not intended to replace human psychiatric diagnosis, this application serves as a powerful, objective assistive tool. It empowers therapists to pinpoint specific moments of hidden distress during telehealth sessions, ultimately facilitating more accurate diagnoses, better-informed therapy progression tracking, and proactive intervention in high-stress environments.

8.2 FUTURE ENHANCEMENTS

While the current iteration of the Fusion Multi-model 'State-of-Mind' Prediction App establishes a robust foundational architecture, several future enhancements can elevate the system from an emotion-tracking tool to a comprehensive diagnostic medical device.

- **Integration of Natural Language Processing (NLP):** Currently, the audio pipeline strictly analyzes how a patient speaks (prosody and tone). Integrating an Automatic Speech Recognition (ASR) module to transcribe the speech to text would allow for Semantic Sentiment Analysis. Analyzing the actual linguistic content in tandem with the visual and acoustic data would create a truly tri-modal system, providing deep context regarding the triggers of emotional volatility.
- **Physiological Sensor Integration:** To further objectify the assessment, a future iteration could incorporate data from wearable devices. Integrating real-time biometrics such as Heart Rate Variability (HRV) or Galvanic Skin Response (GSR) would introduce an autonomic nervous system modality. This biological data is entirely involuntary and would serve as the ultimate ground-truth metric for detecting spikes in anxiety or stress.
- **Dynamic Weighting Optimization:** The current fusion algorithm utilizes static comparisons to determine congruence. Future development should implement a dynamic, confidence-based weighting system. For example, if the visual model registers low confidence because the patient has turned their face away from the camera, the system should automatically dynamically increase the weight of the auditory model's prediction to maintain continuous analytical accuracy.
- **Advanced Diagnostic Pathology Prediction:** The ultimate goal is to transition the system from basic "Emotion Detection" to actual "Pathology Prediction". By aggregating the multi-modal timeline data over weeks or months of therapy sessions, predictive machine learning models could be trained to identify long-term trends. The system could flag specific clinical risks, such as analyzing speech pace and sentence completion rates to trigger early-warning alerts for a "High likelihood of a Depressive or Manic Episode".
- **Explainable AI (XAI) Integration:** To increase clinical trust in the neural network's predictions, integrating Explainable AI frameworks like SHAP (SHapley Additive exPlanations) or Grad-CAM is recommended. Generating heatmaps that visually highlight which specific facial muscles or which exact audio frequencies triggered a "Fearful" or "Sad" classification would provide therapists with transparent, interpretable data to support their clinical decisions.

9. APPENDICES

9.1 PLAGIARISM CERTIFICATE

Dupli Checker

Date: 05-03-2026

Plagiarism Scan Report

0%
Plagiarism

0%
Exact Match

0%
Partial Match

100%
Unique

Words	274
Characters	2191
Sentences	13
Paragraphs	1
Read Time	2 minute(s)
Speak Time	2 minute(s)

Content Checked For Plagiarism

Traditional psychopathology assessments rely heavily on subjective clinical interviews, which are often susceptible to patient bias and the conscious masking of true emotional states. To address the need for objective, continuous emotional monitoring, this project presents a Multi-modal Psychopathology Assessment system designed for robust state-of-mind prediction. The proposed architecture fuses visual and acoustic modalities to provide a comprehensive evaluation of a subject's affective state, mitigating the false positives inherent in single-modality systems. The visual channel employs a custom 2D Convolutional Neural Network (CNN) trained on the FER-2013 dataset to classify seven distinct facial expressions extracted from video frames. Concurrently, the auditory channel utilizes an advanced, hybrid 1D CNN-LSTM network trained on the RAVDESS dataset. Addressing early developmental limitations, the robust auditory model incorporates targeted data augmentation—such as noise injection, pitch shifting, and time stretching—and utilizes Mel-frequency cepstral coefficients (MFCCs) to capture long-term temporal and prosodic dependencies across eight acoustic emotion classes. Furthermore, the system integrates a K-Means clustering algorithm for speaker diarization, allowing the application to isolate and track emotions for specific individuals in multi-speaker environments. These dual independent streams are synchronized within a Streamlit-based web application. The integration hub utilizes a late-fusion algorithmic strategy, actively comparing synchronous visual and auditory predictions to calculate emotional congruence and flag incongruent affect (e.g., a smiling face paired with a stressed voice). The final output is rendered through an interactive, browser-based multimodal player and dynamic Plotly visualizations. Ultimately, this platform equips mental health professionals with detailed, chronological insights regarding a patient's emotional fluctuations, functioning as an impartial clinical decision tool during telehealth consultations and enabling long-term psychological, psychiatric and mental health tracking to support comprehensive patient wellness.

Matched Source

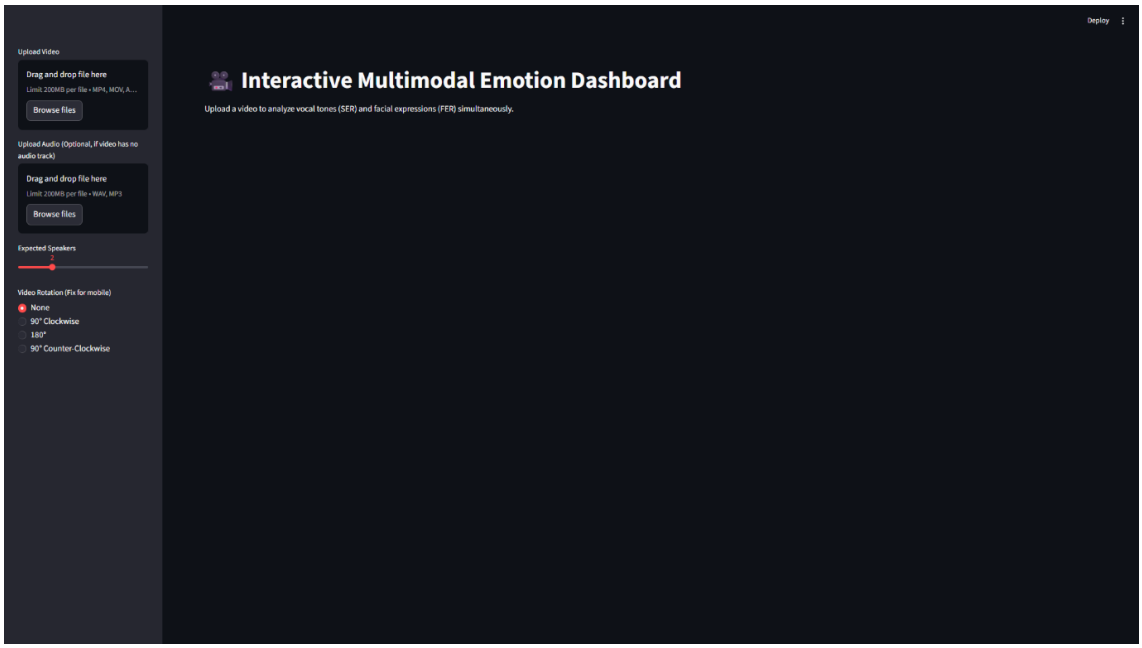
Congratulations !

No Plagiarism Found

Check By: Dupli Checker

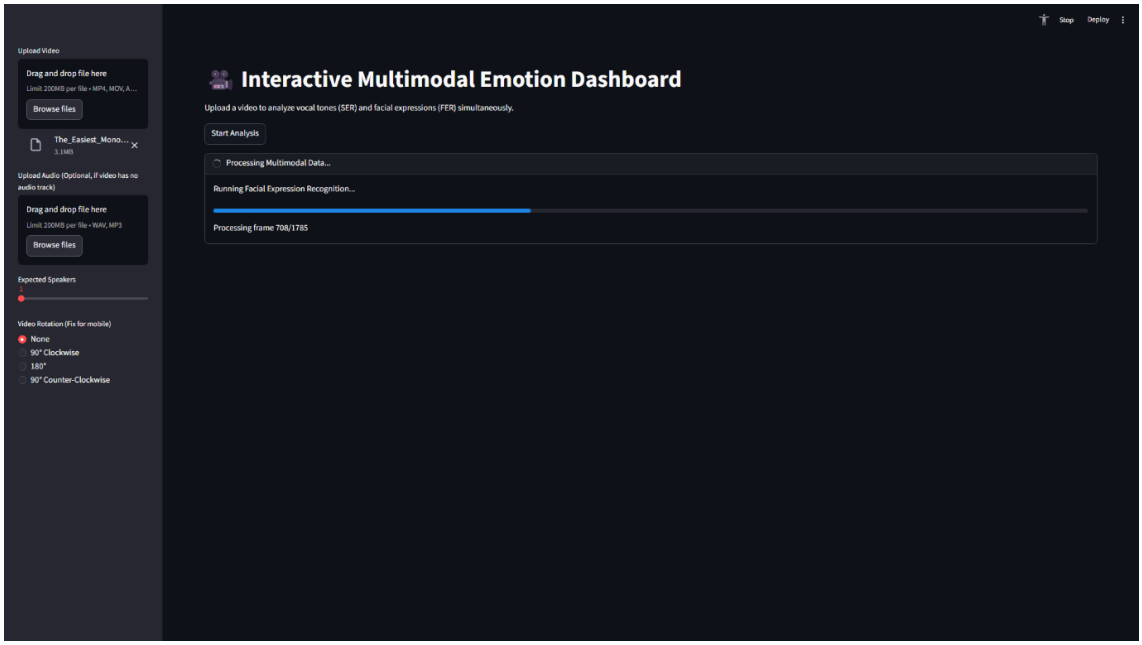
SCREENSHOT 9.1.1

9.2 SCREENSHOTS



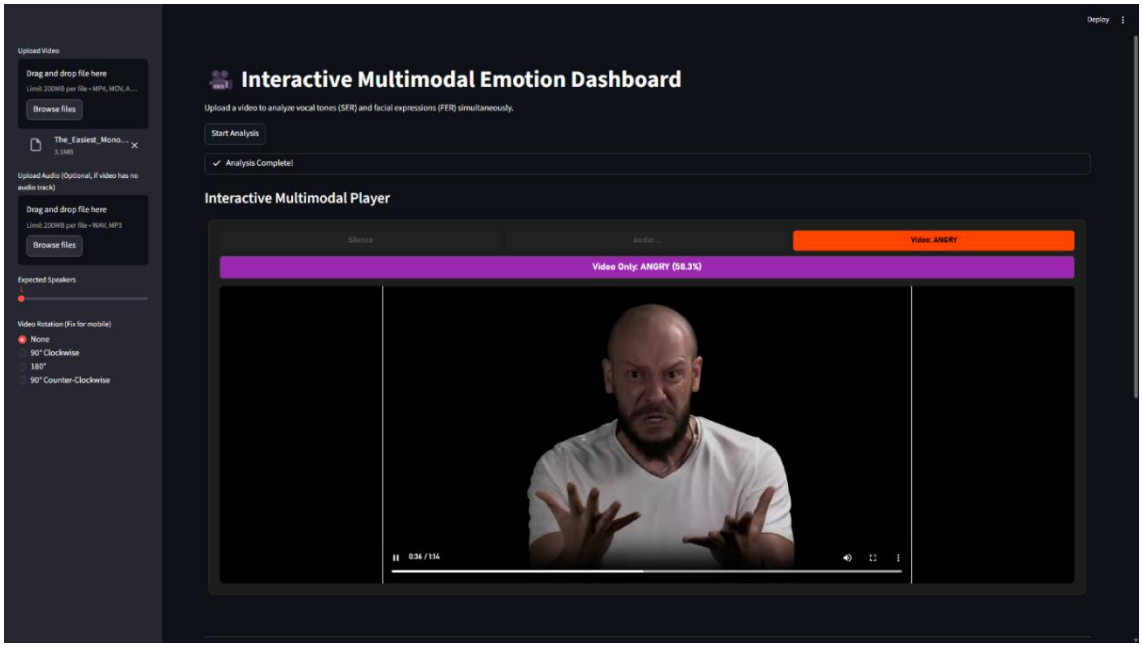
SCREENSHOT 9.2.1 - INITIAL DASHBOARD

This image displays the default landing page of the Streamlit web application. The left sidebar contains the primary user controls, including file uploaders for video and optional separate audio files. It also displays the adjustable parameters for the expected number of speakers (for diarization) and video rotation correction for mobile-recorded media.



SCREENSHOT 9.2.2- ACTIVE PROCESSING STATE

This image demonstrates the system's multithreaded processing phase after a video (The_Easiest_Mono...) has been uploaded. The main panel displays a progress bar indicating that the Facial Expression Recognition (FER) module is actively extracting and analyzing frames (showing frame 708 out of 1785).



SCREENSHOT 9.2.3- MULTIMODAL PLAYER

This captures the custom HTML and JavaScript video player in action during inference. The video displays a subject exhibiting a highly expressive state. The dynamic overlay above the video accurately classifies the frame, displaying "Video: ANGRY". The main fusion banner shows "Video Only: ANGRY (58.3%)" with a purple background, indicating that no concurrent active speech was detected at this exact timestamp (as noted by the "Silence" indicator in the audio box).

SCREENSHOT 9.2.4- MODAL FUSION TIMELINE

This screenshot displays how the Plotly visual reports are embedded within the dashboard interface after the video processing completes. It highlights the "Video Detail" subsection, demonstrating how clinicians can scroll through the interactive graphs alongside the video player to pinpoint specific moments of distress.

SCREENSHOT 9.2.5 - FULLSCREEN EMOTION SCATTER PLOT

This image showcases a detailed, full-width Plotly scatter plot mapping the subject's emotional states over time. The y-axis categorizes the seven distinct emotions, while the x-axis tracks the video timeline in seconds. Each data point's color corresponds to a specific emotion, providing a clear visual representation of emotional clusters and volatility.

9.3 SOURCE CODE

9.3.1 app.py code

```
# importing necessary libraries
import streamlit as st
import streamlit.components.v1 as components
import os
import numpy as np
import librosa
import tensorflow as tf
from tensorflow.keras.models import load_model
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
import pandas as pd
import plotly.express as px
import plotly.graph_objects as go
import tempfile
import cv2
import base64
import json
import time

# Use the directory where the script is located as the base path
BASE_PATH = os.path.dirname(os.path.abspath(__file__))
projectroot = os.path.dirname(BASE_PATH)
# Paths relative to the script location (Root of workspace)
SER_MODEL_PATH = os.path.join(projectroot, "models", "ser_model.h5")
SER_CLASSES_PATH = os.path.join(projectroot, "models", "emotion_classes.npy")
```

```
# FER Model path (in subdirectory)
FER_MODEL_PATH = os.path.join(projectroot, "models","fer_model.h5")

# Audio Config
SAMPLE_RATE = 22050
DURATION = 3.0
N_MFCC = 40
N_MELS = 128
HOP_LENGTH = 512
N_FFT = 2048

# FER Classes (Fixed)
FER_CLASSES = ['Angry', 'Disgust', 'Fear', 'Happy', 'Neutral', 'Sad', 'Surprise']

# Page Config
st.set_page_config(page_title="Multimodal Emotion Analysis", layout="wide")

# --- MODEL LOADING ---
@st.cache_resource
def load_models():
    """Loads SER and FER models."""
    models = {}

    # Load SER
    if os.path.exists(SER_MODEL_PATH):
        try:
            models['ser'] = load_model(SER_MODEL_PATH)
            models['ser_type'] = 'legacy' # Keep legacy preprocessing for my_audio_emotion_model.h5
            models['ser_classes'] = np.load(SER_CLASSES_PATH)
        except Exception as e:
```

```
        st.error(f"Failed to load SER model: {e}")

    # Load FER
    if os.path.exists(FER_MODEL_PATH):
        try:
            models['fer'] = load_model(FER_MODEL_PATH)
            # Load Haar Cascade
            models['face_cascade'] = cv2.CascadeClassifier(cv2.data.haarcascades + 'haarcascade_frontalface_default.xml')
        except Exception as e:
            st.error(f"Failed to load FER model: {e}")

    return models

# --- AUDIO PROCESSING FUNCTIONS ---
def extract_features_for_diarization(y, sr):
    mfccs = librosa.feature.mfcc(y=y, sr=sr, n_mfcc=20)
    mfcc_mean = np.mean(mfccs, axis=1)
    mfcc_std = np.std(mfccs, axis=1)
    cent = librosa.feature.spectral_centroid(y=y, sr=sr)
    bw = librosa.feature.spectral_bandwidth(y=y, sr=sr)
    rolloff = librosa.feature.spectral_rolloff(y=y, sr=sr)

    features = np.concatenate([
        mfcc_mean, mfcc_std,
        [np.mean(cent), np.std(cent), np.mean(bw), np.std(bw), np.mean(rolloff), np.std(rolloff)]
    ])
    return features

def preprocess_for_emotion_advanced(y, sr):
```

```
    mel_spec = librosa.feature.melspectrogram(y=y, sr=sr, n_mels=N_MELS, n_fft=N_FFT, hop_length=HOP_LENGTH)
    mel_spec_db = librosa.power_to_db(mel_spec, ref=np.max)
    return np.expand_dims(mel_spec_db.T, axis=-1)

def preprocess_for_emotion_legacy(y, sr):
    mfccs = librosa.feature.mfcc(y=y, sr=sr, n_mfcc=N_MFCC)
    return mfccs.T

def process_audio(file_path, model, classes, model_type, num_speakers=2):
    try:
        y, sr = librosa.load(file_path, sr=SAMPLE_RATE)
    except Exception as e:
        st.warning(f"Could not extract audio (likely missing ffmpeg): {e}")
        return pd.DataFrame()

    intervals = librosa.effects.split(y, top_db=25, frame_length=2048, hop_length=512)
    segments_data = []
    diarization_features = []

    target_len = int(DURATION * SAMPLE_RATE)
    stride_samples = int(1.0 * sr)

    for start_idx, end_idx in intervals:
        segment_audio = y[start_idx:end_idx]
        if len(segment_audio) / sr < 0.5: continue

        start_time = start_idx / sr
        end_time = end_idx / sr

        # Diarization
```

```
d_feat = extract_features_for_diarization(segment_audio, sr)
diarization_features.append(d_feat)

# Emotion
predictions = []
if len(segment_audio) > target_len:
    for i in range(0, len(segment_audio) - target_len + 1, stride_samples):
        chunk = segment_audio[i : i + target_len]
        inp = preprocess_for_emotion_advanced(chunk, sr) if model_type == 'advanced' else preprocess_for_emotion_legacy(chunk, sr)
        predictions.append(model.predict(np.expand_dims(inp, axis=0), verbose=0)[0])

if not predictions:
    padded = np.pad(segment_audio, (0, max(0, target_len - len(segment_audio))), 'constant')[:target_len]
    inp = preprocess_for_emotion_advanced(padded, sr) if model_type == 'advanced' else preprocess_for_emotion_legacy(padded, sr)
    predictions.append(model.predict(np.expand_dims(inp, axis=0), verbose=0)[0])

avg_pred = np.mean(predictions, axis=0)
max_idx = np.argmax(avg_pred)

segments_data.append({
    "Start": start_time,
    "End": end_time,
    "Emotion": classes[max_idx],
    "Confidence": float(avg_pred[max_idx]),
    "Duration": end_time - start_time,
    "Type": "Audio"
})
```

```
    if not segments_data: return pd.DataFrame()

    # Diarization Clustering
    if len(diarization_features) >= num_speakers and num_speakers > 1:
        try:
            scaler = StandardScaler()
            X_diar = scaler.fit_transform(np.array(diarization_features))
            kmeans = KMeans(n_clusters=num_speakers, random_state=42, n_init=20)
            speakers = kmeans.fit_predict(X_diar)
             # Smoothing
            if len(speakers) > 2:
                for j in range(1, len(speakers) - 1):
                    if speakers[j-1] == speakers[j+1] and speakers[j] != speakers[j-1]:
                        speakers[j] = speakers[j-1]
        except:
            speakers = [0] * len(segments_data)
    else:
        speakers = [0] * len(segments_data)

    for i, data in enumerate(segments_data):
        data["Speaker"] = f"Speaker {speakers[i] + 1}"

    return pd.DataFrame(segments_data)

# --- VIDEO PROCESSING FUNCTIONS ---
def process_video_fer(video_path, fer_model, face_cascade, rotation_mode="None"):
    cap = cv2.VideoCapture(video_path)
    fps = cap.get(cv2.CAP_PROP_FPS)
    total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
    duration = total_frames / fps if fps > 0 else 0
```

```
fer_data = []

# Process every 0.5 seconds
step = int(fps * 0.5) if fps > 0 else 15

prog_bar = st.progress(0)
status_text = st.empty()

frame_idx = 0
while True:
    ret, frame = cap.read()
    if not ret: break

    # Apply rotation if requested
    if rotation_mode == "90° Clockwise":
        frame = cv2.rotate(frame, cv2.ROTATE_90_CLOCKWISE)
    elif rotation_mode == "180°":
        frame = cv2.rotate(frame, cv2.ROTATE_180)
    elif rotation_mode == "90° Counter-Clockwise":
        frame = cv2.rotate(frame, cv2.ROTATE_90_COUNTERCLOCKWISE)

    if frame_idx % step == 0:
        status_text.text(f"Processing frame {frame_idx}/{total_frames}")
        gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
        faces = face_cascade.detectMultiScale(gray, 1.3, 5)

        if len(faces) > 0:
            # Find largest face
            max_area = 0
            best_face = faces[0]
            for (x, y, w, h) in faces:
```

```
            if w*h > max_area:
                max_area = w*h
                best_face = (x, y, w, h)

        x, y, w, h = best_face
        roi_gray = gray[y:y+h, x:x+w]
        try:
            roi_gray = cv2.resize(roi_gray, (48, 48))
            img_pixels = roi_gray.astype('float32') / 255.0
            img_pixels = np.expand_dims(img_pixels, axis=0)
            img_pixels = np.expand_dims(img_pixels, axis=-1)

            probs = fer_model.predict(img_pixels, verbose=0)[0]
            max_idx = np.argmax(probs)
            emotion = FER_CLASSES[max_idx]
            confidence = float(probs[max_idx])

            fer_data.append({
                "Time": frame_idx / fps,
                "Emotion": emotion,
                "Confidence": confidence,
                "Type": "Video"
            })
        except Exception as e:
            pass

    frame_idx += 1
    if frame_idx % 50 == 0:
        prog_bar.progress(min(1.0, frame_idx / max(1, total_frames)))

cap.release()
```

```
    prog_bar.empty()
    status_text.empty()
    return pd.DataFrame(fer_data)

# --- INTERACTIVE PLAYER HTML ---
def get_multimodal_player_html(video_path, df_audio, df_video):
    """
    Generates an HTML video player with synchronized Emotion/Speaker display and Fusion logic.
    """
    with open(video_path, "rb") as f:
        video_bytes = f.read()
    video_base64 = base64.b64encode(video_bytes).decode()
    # Detect mime type roughly
    ext = os.path.splitext(video_path)[1].lower().replace('.', '')
    mime = f"video/{ext}" if ext != 'mkv' else 'video/mp4'

    # Convert DataFrames to JSON for JS (Ensure native types)
    audio_records = []
    if not df_audio.empty:
        # Explicitly cast to float/str to avoid numpy serialization issues
        temp_df = df_audio.copy()
        temp_df['Start'] = temp_df['Start'].astype(float)
        temp_df['End'] = temp_df['End'].astype(float)
        temp_df['Confidence'] = temp_df['Confidence'].astype(float)
        temp_df['Speaker'] = temp_df['Speaker'].astype(str)
        temp_df['Emotion'] = temp_df['Emotion'].astype(str)
        audio_records = temp_df[['Start', 'End', 'Speaker', 'Emotion', 'Confidence']].to_dict(orient='records')

    video_records = []
```

```
if not df_video.empty:
    temp_df = df_video.copy()
    temp_df['Time'] = temp_df['Time'].astype(float)
    temp_df['Confidence'] = temp_df['Confidence'].astype(float)
    temp_df['Emotion'] = temp_df['Emotion'].astype(str)
    video_records = temp_df[['Time', 'Emotion', 'Confidence']].to_dict(orient='records')

json_audio = json.dumps(audio_records)
json_video = json.dumps(video_records)

html = f"""
<div style="background-color: #1E1E1E; padding: 20px; border-radius: 10px; margin-bottom: 20px; border: 1px solid #333; font-family: sans-serif; text-align: center;">
    <div style="display: grid; grid-template-columns: 1fr 1fr 1fr; gap: 10px; margin-bottom: 10px;">
        <div id="speaker-box" style="padding: 10px; border-radius: 6px; background-color: #333; color: #888; text-align: center; font-weight: bold; font-size: 0.9em;">Unknown</div>
        <div id="audio-emo-box" style="padding: 10px; border-radius: 6px; background-color: #333; color: #888; text-align: center; font-weight: bold; font-size: 0.9em;">Audio: ...</div>
        <div id="video-emo-box" style="padding: 10px; border-radius: 6px; background-color: #333; color: #888; text-align: center; font-weight: bold; font-size: 0.9em;">Video: ...</div>
    </div>
    <div id="fusion-box" style="padding: 10px; margin-bottom: 15px; border-radius: 6px; background-color: #333; color: #888; text-align: center; font-weight: bold; font-size: 1.1em; border: 1px solid #555;">Fused: ...</div>

    <div style="width: 100%; display: flex; justify-content: center; background: black; border-radius: 8px; overflow: hidden;">
        <video id="video-player" controls style="width: auto; max-width: 100%; height: auto; max-height: 75vh;">
            <source src="data:{mime}; base64,{video_base64}" type="{mime}">
            Your browser does not support the video element.
```

```
    </video>
</div>

<script>
    const audioSegments = {json_audio};
    const videoFrames = {json_video};

    const video = document.getElementById('video-player');
    const speakerBox = document.getElementById('speaker-box');
    const audioBox = document.getElementById('audio-emo-box');
    const videoBox = document.getElementById('video-emo-box');
    const fusionBox = document.getElementById('fusion-box');

    const colors = {{
        'neutral': '#D3D3D3', 'calm': '#ADD8E6', 'happy': '#FFD700',
        'sad': '#4682B4', 'angry': '#FF4500', 'fearful': '#800080',
        'disgust': '#008000', 'surprised': '#FFA500',
        'Neutral': '#D3D3D3', 'Happy': '#FFD700', 'Sad': '#4682B4',
        'Angry': '#FF4500', 'Fear': '#800080', 'Disgust': '#008000',
        'Surprise': '#FFA500'
    }};

    function updatePlayer() {{
        const t = video.currentTime;

        let currAudio = null;
        let currVideo = null;

        // 1. Audio Update
        const activeAudio = audioSegments.find(s => t >= s.Start && t < s.End);
        if (activeAudio) {{
```

```
currAudio = activeAudio.Emotion.toLowerCase();
// Normalize SER labels to match FER for fusion comparison
if (currAudio === 'fearful') currAudio = 'fear';
if (currAudio === 'surprised') currAudio = 'surprise';
if (currAudio === 'calm') currAudio = 'neutral'; // Optional: map calm to neutral

speakerBox.innerText = activeAudio.Speaker; speakerBox.style.backgroundColor = '#444';
speakerBox.style.color = 'white';

audioBox.innerText = "Audio: " + activeAudio.Emotion.toUpperCase();
audioBox.style.backgroundColor = colors[currAudio] || '#333';
audioBox.style.color = '#000';
}} else {{
speakerBox.innerText = "Silence";
speakerBox.style.backgroundColor = '#222';
speakerBox.style.color = '#555';

audioBox.innerText = "Audio: ...";
audioBox.style.backgroundColor = '#222';
audioBox.style.color = '#555';
}}

// 2. Video Update
// Find closest frame within 1.5s window
let bestFrame = null;
let minDiff = 1.5;

for (let i = 0; i < videoFrames.length; i++) {{
const diff = Math.abs(videoFrames[i].Time - t);
```

```
            if (diff < minDiff) {{
                minDiff = diff;
                bestFrame = videoFrames[i];
            }}
        }}

        let videoEmoStr = null;
        if (bestFrame) {{
            currVideo = bestFrame;
            videoEmoStr = currVideo.Emotion.toLowerCase();
            videoBox.innerText = "Video: " + currVideo.Emotion.toUpperCase();
            videoBox.style.backgroundColor = colors[currVideo.Emotion] || '#333';
            videoBox.style.color = '#000';
        }} else {{
            videoBox.innerText = "Video: ...";
            videoBox.style.backgroundColor = '#222';
            videoBox.style.color = '#555';
        }}

        // 3. Fusion Logic
        if (currAudio && videoEmoStr) {{
            const audioConf = (activeAudio.Confidence * 100).toFixed(1);
            const videoConf = (currVideo.Confidence * 100).toFixed(1);
            if (currAudio === videoEmoStr) {{
                fusionBox.innerText = `Fused: ${{currAudio.toUpperCase()}} (Audio: ${{audioConf}}%, Video: ${{videoConf}}%)`;
                fusionBox.style.backgroundColor = '#4CAF50'; // Green match
                fusionBox.style.color = 'white';
            }} else {{
                fusionBox.innerText = `Mixed: Audio ${{currAudio.toUpperCase()}} (${{audioConf}}%) vs Video ${{videoEmoStr.toUpperCase()}} (${{videoConf}}%)`;
```

```
                fusionBox.style.backgroundColor = '#FF9800'; // Orange conflict
                fusionBox.style.color = 'black';
            }}
        }} else if (currAudio) {{
            const audioConf = (activeAudio.Confidence * 100).toFixed(1);
            fusionBox.innerText = `Audio Only: ${{currAudio.toUpperCase()}} (${{audioConf}}%)`;
            fusionBox.style.backgroundColor = '#2196F3'; // Blue (Audio only)
            fusionBox.style.color = 'white';
        }} else if (videoEmoStr) {{
            const videoConf = (currVideo.Confidence * 100).toFixed(1);
            fusionBox.innerText = `Video Only: ${{videoEmoStr.toUpperCase()}} (${{videoConf}}%)`;
            fusionBox.style.backgroundColor = '#9C27B0'; // Purple (Video only)
            fusionBox.style.color = 'white';
        }} else {{
            fusionBox.innerText = "Fused: ...";
            fusionBox.style.backgroundColor = '#333';
            fusionBox.style.color = '#888';
        }}
    }}

    video.ontimeupdate = updatePlayer;
  </script>
</div>
"""
return html


# --- UI ---
st.title(" Interactive Multimodal Emotion Dashboard")
st.markdown("Upload a video to analyze vocal tones (SER) and facial expressions (FER) simultaneously.")
```

```
models = load_models()
if not models:
    st.error("No models found. Please check paths.")
    st.stop()

uploaded_file = st.sidebar.file_uploader("1. Upload Video", type=['mp4', 'mov', 'avi', 'mkv'])
uploaded_audio = st.sidebar.file_uploader("2. Upload Audio (Optional, if video has no audio track)", type=['wav', 'mp3'])
num_speakers = st.sidebar.slider("Expected Speakers", 1, 5, 2)
rotation_mode = st.sidebar.radio("Video Rotation (Fix for mobile)", ["None", "90° Clockwise", "180°", "90° Counter-Clockwise"])

if uploaded_file:
    # Save temp file
    tfile = tempfile.NamedTemporaryFile(delete=False, suffix=os.path.splitext(uploaded_file.name)[1])
    tfile.write(uploaded_file.read())
    tfile.close() # CRITICAL: Close file to ensure it's fully written/flush buffers
    video_path = tfile.name

    # Determine Audio Source
    audio_path = video_path
    if uploaded_audio:
        afile = tempfile.NamedTemporaryFile(delete=False, suffix=os.path.splitext(uploaded_audio.name)[1])
        afile.write(uploaded_audio.read())
        afile.close() # Close audio file too
        audio_path = afile.name
        st.sidebar.info("Using separate audio file for SER analysis.")

    # Initialize session state
```

```
if 'df_audio' not in st.session_state:
    st.session_state.df_audio = pd.DataFrame()
if 'df_video' not in st.session_state:
    st.session_state.df_video = pd.DataFrame()
if 'analyzed' not in st.session_state:
    st.session_state.analyzed = False

if st.button("Start Analysis"):
    st.session_state.analyzed = False
    with st.status("Processing Multimodal Data...", expanded=True) as status:

        # 1. Video Analysis
        st.write("Running Facial Expression Recognition...")
        df_video = pd.DataFrame()
        if 'fer' in models:
            # Pass rotation mode here
            df_video = process_video_fer(video_path, models['fer'], models['face_cascade'], rotation_mode)
            st.session_state.df_video = df_video
            if len(df_video) > 0:
                st.write(f" Processed {len(df_video)} video frames.")
            else:
                st.error(" Processed 0 frames. Video might be corrupted or format unsupported by OpenCV.")

        # 2. Audio Analysis
        st.write("Running Speech Emotion Recognition...")
        df_audio = pd.DataFrame()
        if 'ser' in models:
            # Try to process
            df_audio = process_audio(audio_path, models['ser'], models['ser_classes'], models['ser_type'], num_speakers)
```

```
            st.session_state.df_audio = df_audio

            if not df_audio.empty:
                st.write(f" Detected {len(df_audio)} audio segments.")
            else:
                if not uploaded_audio:
                    st.error(" No audio detected! This system lacks FFmpeg to extract audio from video. Please upload the .wav audio file separately in the sidebar.")
                else:
                    st.warning("Audio processing failed on the uploaded file.")

        st.session_state.analyzed = True
        status.update(label="Analysis Complete!", state="complete", expanded=False)

    if st.session_state.analyzed:
        df_audio = st.session_state.df_audio
        df_video = st.session_state.df_video

        # --- INTERACTIVE PLAYER (TOP) ---
        st.subheader("Interactive Multimodal Player")

        # Check file size before attempting base64 encoding (approx 50MB limit for smooth experience)
        file_size_mb = os.path.getsize(video_path) / (1024 * 1024)

        # Increased limit to 200MB and rely on browser handling
        if file_size_mb > 200:
            st.warning(f" Video is large ({file_size_mb:.1f} MB). Using standard player (No interactive overlay).")
            st.video(video_path)
        else:
            try:
```

```
        # Using a taller iframe for mobile-format support
        html_code = get_multimodal_player_html(video_path, df_audio, df_video)
        components.html(html_code, height=750, scrolling=False)
    except Exception as e:
        st.error(f"Could not load interactive player: {e}")
        st.video(video_path)

# --- VISUALIZATION ---
st.divider()
st.subheader("Multimodal Fusion Timeline")

if not df_audio.empty and not df_video.empty:
    # Create a unified timeline
    fig = go.Figure()

    # Add Audio Traces
    fig.add_trace(go.Scatter(
        x=df_audio['Start'],
        y=[1]*len(df_audio),
        mode='markers',
        marker=dict(size=10, symbol='circle', color=df_audio['Emotion'].map({
                'neutral': '#D3D3D3', 'calm': '#ADD8E6', 'happy': '#FFD700',
                'sad': '#4682B4', 'angry': '#FF4500', 'fearful': '#800080',
                'disgust': '#008000', 'surprised': '#FFA500'
            })),
        name='Audio Emotion',
        text=df_audio['Emotion'],
        hovertemplate="Audio: %{text}<br>Time: %{x:.2f}s"
    ))

    # Add Video Traces
```

```
    fig.add_trace(go.Scatter(
        x=df_video['Time'],
        y=[2]*len(df_video),
        mode='markers',
        marker=dict(size=8, symbol='square', color=df_video['Emotion'].map({
                'Neutral': '#D3D3D3', 'Happy': '#FFD700', 'Sad': '#4682B4',
                'Angry': '#FF4500', 'Fear': '#800080', 'Disgust': '#008000',
                'Surprise': '#FFA500'
            })),
        name='Video Emotion',
        text=df_video['Emotion'],
        hovertemplate="Video: %{text}<br>Time: %{x:.2f}s"
    ))

    fig.update_layout(
        title="Audio vs Video Emotion Alignment",
        yaxis=dict(
            tickmode='array',
            tickvals=[1, 2],
            ticktext=['Audio', 'Video'],
            range=[0.5, 2.5]
        ),
        xaxis_title="Time (s)",
        height=300
    )
    st.plotly_chart(fig, use_container_width=True)

col1, col2 = st.columns(2)

with col1:
    if not df_audio.empty:
```

```
        st.write("#### Audio Detail")
        fig_audio = px.scatter(
            df_audio, x="Start", y="Emotion", color="Emotion",
            size="Confidence", hover_data=["Speaker", "Duration"],
            color_discrete_map={
                'neutral': '#D3D3D3', 'calm': '#ADD8E6', 'happy': '#FFD700',
                'sad': '#4682B4', 'angry': '#FF4500', 'fearful': '#800080',
                'disgust': '#008000', 'surprised': '#FFA500'
            }
        )
        st.plotly_chart(fig_audio, use_container_width=True)

    with col2:
        if not df_video.empty:
            st.write("#### Video Detail")
            fig_video = px.scatter(
                df_video, x="Time", y="Emotion", color="Emotion",
                size="Confidence",
                color_discrete_map={
                    'Neutral': '#D3D3D3', 'Happy': '#FFD700', 'Sad': '#4682B4',
                    'Angry': '#FF4500', 'Fear': '#800080', 'Disgust': '#008000',
                    'Surprise': '#FFA500'
                }
            )
            st.plotly_chart(fig_video, use_container_width=True)
```

9.4 USER DOCUMENTATION

9.4.1 Introduction

The Fusion Multi-modal 'State-of-Mind' Prediction App is an interactive web-based clinical decision support tool designed for therapists and researchers. It processes pre-recorded session videos to simultaneously analyze facial micro-expressions and vocal intonations (prosody). By cross-referencing these two streams, the system provides objective, timeline-based insights into a subject's emotional volatility and flags incongruent emotional states (e.g., masking stress with a smile).

9.4.2 System Requirements

Before operating the software, ensure your system meets the following prerequisites:

- Operating System: Windows 10/11, macOS, or Linux.
- Software: Python 3.8 or higher.
- System Packages: FFmpeg must be installed on your host machine to allow the system to extract audio tracks from video files.
- Hardware: A multi-core CPU and a minimum of 8GB RAM are recommended for smooth video processing. A dedicated GPU (NVIDIA) is recommended but not strictly required for inference.

9.4.3 Installation and Setup

- Acquire the Source Code: Clone the project repository to your local machine.
- Environment Setup: It is highly recommended to create a Python virtual environment to prevent dependency conflicts.
- Install Dependencies: Open your terminal or command prompt, navigate to the project root folder, and install the required libraries by executing: pip install -r requirements.txt.
- Model Verification: Ensure the pre-trained neural network weights are located in their correct directories:
- Audio Model: models/ser _model.h5

- Audio Encoders: models/emotion_classes.npy
- Video Model: models/fer_model.h5

9.4.4 Launching the Application

- To start the dashboard, open your terminal, navigate to the directory containing the application script, and run the following command: streamlit run video_emotion_dashboard.py
- This will initialize the local server and automatically open the dashboard in your default web browser (typically at http://localhost:8501).

9.4.5 Step-by-Step Usage Guide

- **Step 1: Uploading Media**

  Upon launching the dashboard, navigate to the left-hand sidebar to input your media files.

  - Upload Video: Drag and drop your primary session recording into the "1. Upload Video" box. Supported formats include .mp4, .mov, .avi, and .mkv.

  - Upload Audio (Optional): If your video file does not contain an embedded audio track, or if the system warns you that it cannot extract the audio, use the "2. Upload Audio" box to provide a separate .wav or .mp3 file for the acoustic analysis.

- **Step 2: Configuring Parameters**

  Below the file uploaders, configure the processing parameters to match your media:

  - Expected Speakers: Adjust this slider (ranging from 1 to 5) to indicate how many distinct voices are in the recording. This is critical for the speaker diarization module, which will attempt to separate the patient's voice from the clinician's.

- Video Rotation: If your video was recorded vertically on a mobile device and appears sideways, select the appropriate rotational fix (e.g., "90° Clockwise") so the facial recognition model can accurately detect horizontal facial features.

- **Step 3: Executing the Analysis**

  Once your files are uploaded and parameters are set, click the "Start Analysis" button.

  - The system will display a progress bar as it extracts frames and audio chunks.
  - It processes video frames every 0.5 seconds and audio segments in 3.0-second buffers. Please be patient, as this deep learning inference may take a few minutes depending on the length of the video.

9.4.6 Interpreting the Results

After processing is complete, the dashboard will render several interactive components to help you analyze the session.

1. **The Interactive Multimodal Player**

   Located at the top of the results section, this custom video player synchronizes the neural network predictions with the video playback. Speaker Box: Displays which clustered speaker (e.g., "Speaker 1") is currently talking.

   - **Audio & Video Boxes:** Display the isolated emotional predictions for the vocal tone and facial expression at that exact timestamp.
   - **The Fusion Banner:** This is the core diagnostic tool. It compares the audio and video streams:
   - **Green (Fused):** Indicates emotional congruence (both modalities detect the same emotion, e.g., Audio: Happy and Video: Happy).
   - **Orange (Mixed):** Indicates a conflict or emotional masking (e.g., Audio indicates Anger, but Video indicates Happiness).
   - **Blue/Purple:** Indicates that only one modality successfully detected an emotion at that timestamp (Audio Only or Video Only).

Note: If your uploaded video exceeds 200MB, the system will automatically disable this custom interactive player to prevent browser crashes and will revert to a standard video player. You can still rely on the charts below.

2. **Analytical Scatter Plots**

   Scroll down below the video player to view the detailed session reports generated via Plotly.

   - **Multimodal Fusion Timeline:** This unified chart overlays both Audio (circle markers) and Video (square markers) emotional states across the entire duration of the video, allowing you to easily spot trends and clusters of distress.

   - **Audio / Video Detail Plots:** These separate charts provide granular data. The size of each plotted point represents the neural network's confidence score for that prediction. You can hover your mouse cursor over any data point to view specific details, such as the exact timestamp, confidence percentage, and the identified Speaker ID.

9.5 GLOSSARY

1. **Affective Computing:** A field of study that focuses on analyzing non-verbal cues to introduce objective measurement into emotional and psychological assessments.

2. **CNN (Convolutional Neural Network):** A deep learning architecture heavily utilized as a primary feature extractor. The system utilizes a 2D CNN to classify visual emotions and a 1D CNN for processing acoustic features.

3. **Congruent Affect:** A state determined during data fusion when both visual and auditory predictions match conceptually (e.g., a sad face paired with a sad voice).

4. **FER (Facial Emotion Recognition):** A computer vision pipeline designed to categorize facial expressions into seven standard emotional states: Angry, Disgust, Fear, Happy, Neutral, Sad, and Surprise.

5. **Haar Cascade Classifier:** An object detection algorithm utilized during visual preprocessing to isolate bounding boxes around human faces and extract the Region of Interest (ROI).

6. **Incongruent Affect:** A flagged "mixed" state occurring when visual and auditory emotional predictions conflict, such as a smiling face paired with a stressed vocal tone.

7. **K-Means Clustering:** An algorithm used in the speaker diarization module to evaluate normalized spectral features and differentiate between distinct vocal signatures in an audio stream.

8. **Late-Fusion (Decision-Level Fusion):** An algorithmic integration strategy that actively compares synchronous visual and auditory predictions to calculate emotional congruence and determine a final state of mind.

9. **Librosa:** A Python-based audio processing library utilized for loading audio waveforms, extracting features like MFCCs, and executing data augmentation functions.

10. **LSTM (Long Short-Term Memory):** A specialized neural network architecture utilized in the auditory model to capture the long-term temporal dependencies of vocal intonation. It relies on an internal memory state regulated by Forget, Input, and Output gates to prevent the vanishing gradient problem.

11. **Masking:** The human tendency, occurring either consciously or subconsciously, to hide true feelings and emotions in social or clinical settings.

12. **MFCCs (Mel-Frequency Cepstral Coefficients):** Extracted prosodic acoustic features used by the neural network to capture long-term temporal and prosodic dependencies for speech emotion classification.

13. **OpenCV:** A computer vision library utilized by the system to handle visual preprocessing tasks, including reading video files, extracting frames, and converting images to grayscale.

14. **Plotly:** A data visualization tool utilized by the application to generate dynamic, interactive scatter plots and timelines mapping a subject's emotional volatility.

15. **Prosody:** Speech tone and involuntary vocal cues related to arousal and valence, which serve as critical indicators for accurate mental state determination.

16. **ROI (Region of Interest):** An isolated, cropped area of a video frame containing the primary subject's face, which is then resized and normalized for neural network analysis.

17. **SER (Speech Emotion Recognition):** An advanced acoustic analysis pipeline that extracts prosodic features to classify vocal tones into distinct emotional categories.

18. **Speaker Diarization:** An analytical module utilizing K-Means clustering to distinguish and track emotional states among multiple speakers, actively separating a patient's voice from a clinician's.

19. **Streamlit:** A Python web framework utilized as the system's front-end for rapidly deploying the localized, interactive dashboard without requiring external server processing.

10. BIBILIOGRAPHY

10.1 Book References

[1] Picard, R. W. (1997). Affective computing. MIT Press

[2] Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep learning. MIT Press

[3] Rabiner, L. R., & Juang, B. H. (1993). Fundamentals of speech recognition. Prentice-Hall.

[4] Ekman, P., & Rosenberg, E. L. (Eds.). (1997). What the face reveals: Basic and applied studies of spontaneous expression using the Facial Action Coding System (FACS). Oxford University Press.

[5] Chollet, F. (2021). *Deep learning with Python* (2nd ed.). Manning Publications.

[6] Calvo, R. A., D'Mello, S., Gratch, J., & Kappas, A. (Eds.). (2015). *The Oxford handbook of affective computing*. Oxford University Press.

[7] Szeliski, R. (2010). *Computer vision: algorithms and applications*. Springer Science & Business Media.

[8] Schuller, B. W., & Batliner, A. (2013). *Computational paralinguistics: emotion, affect and personality in speech and language processing*. Wiley.

[9] Bishop, C. M. (2006). *Pattern recognition and machine learning*. Springer.

[10] Oppenheim, A. V., & Schafer, R. W. (2009). *Discrete-time signal processing* (3rd ed.). Pearson.

[11] Jurafsky, D., & Martin, J. H. (2009). *Speech and language processing* (2nd ed.). Pearson.

10.2 Journal References

[1] Khalil, R. A., Jones, E., Babar, M. I., Jan, T., Zafar, M. H., & Alhassany, T. (2019). Speech emotion recognition using deep learning techniques: A review. IEEE Access, 7, 117327-117345. https://doi.org/10.1109/ACCESS.2019.2936124

[2] Poria, S., Cambria, E., Bajpai, R., & Hussain, A. (2017). A review of affective computing: From unimodal analysis to multimodal fusion. Information Fusion, 37, 98-125. https://doi.org/10.1016/j.inffus.2017.02.003

[3] Zeng, Z., Pantic, M., Roisman, G. I., & Huang, T. S. (2009). A survey of affect recognition methods: Audio, visual, and spontaneous expressions. IEEE Transactions on Pattern Analysis and Machine Intelligence, 31(1), 39-58. https://doi.org/10.1109/TPAMI.2008.52

[4] Zhao, J., Mao, X., & Chen, L. (2019). Speech emotion recognition using deep 1D & 2D CNN LSTM networks. Biomedical Signal Processing and Control, 47, 312-323. https://doi.org/10.1016/j.bspc.2018.08.035

[5] Cummins, N., Scherer, S., Krajewski, J., Schnieder, S., Epps, J., & Quatieri, T. F. (2015). A review of depression and suicide risk assessment using speech analysis. Speech Communication, 71, 10-49. https://doi.org/10.1016/j.specom.2015.03.004

[6] Abbaschian, B. J., Sierra-Sosa, D., & Elmaghraby, A. (2021). Deep Learning Techniques for Speech Emotion Recognition, from Databases to Models. Sensors, 21(4), 1249. https://doi.org/10.3390/s21041249

[7] Ahmed, M. R., Islam, S., D, P., et al. (2021). An Ensemble 1D-CNN-LSTM-GRU Model with Data Augmentation for Speech Emotion Recognition. arXiv. https://doi.org/10.48550/arxiv.2112.05666

[8] Goodfellow, I. J., Erhan, D., Carrier, P. L., et al. (2013). Challenges in Representation Learning: A report on three machine learning contests. arXiv. https://doi.org/10.48550/arxiv.1307.0414